

WHATNEXT VISION MOTORS

Shaping the Future of Mobility with Innovation and Excellence
Salesforce-Based Automotive Dealership and Order Management
System

PROJECT DOCUMENTATION

Detailed implementation, configuration, testing, deployment and source-control report.

TEAM LEADER: MAHALAKSHMI U

TEAM MEMBERS:

ABINAYA S

ELAKIYA C

KIRUTHIGA A

GOWSIK V

COLLEGE: GOVERNMENT COLLEGE OF ENGINEERING, ERODE

GitHub Repository: <https://github.com/Maha-2327/Salesforce.git>

TABLE OF CONTENTS

Section / Module	Page
Document Scope and Implementation Boundary	1
1. Introduction	2
1.1 Project Overview	2
1.2 Project Objectives	3
2. Ideation Phase	5
2.1 Problem Statement Matrix	5
2.2 User Personas & Empathy Map	6
2.3 Brainstorming and Selection	7
3. Requirement Analysis	8
3.1 Customer Journey Map	8
3.2 Functional Requirements	9
3.3 Non-Functional Requirements	10
3.4 Data Flow Architecture	11
4. Technology Stack	12
5. Project Design	13
5.1 Problem-Solution Fit	13

5.2 Solution Architecture	14
6. Project Planning & Scheduling	15
6.1 Work Breakdown Structure	15
6.2 Product Backlog and Sprint Schedule	16
6.3 Sprint Tracker	17
7. Development Environment Setup	18
7.1 Developer Org Signup & Password Setup	18
7.2 Salesforce Org Setup Home	19
8. Data Model Configuration	20
8.1 Custom Objects Creation (Vehicle, Dealer)	20
8.2 Vehicle Fields & Relationships	23
8.3 Dealer, Customer, and Order Data Dictionaries	26
9. Custom Tabs & Lightning Experience App	28
9.1 Tabs Creation	28
9.2 WhatNext Vision Motors Lightning App Setup	29
10. Security Model	32
11. Record-Triggered Flows Implementation	33
11.1 Auto Assign Dealer Flow Configuration	33

11.2 Test Drive Reminder Flow & Scheduled Paths	37
12. Apex Programming & Trigger Handler Pattern	41
12.1 VehicleOrderTriggerHandler Class Implementation	41
12.2 VehicleOrderTrigger Implementation	44
12.3 Out-of-Stock Validation UI Error Trap	45
13. Batch Apex: Automated Order Reconciliation	46
13.1 VehicleOrderBatch Class Implementation	46
14. Schedulable Apex & CRON Automation	48
14.1 VehicleOrderBatchScheduler Class	48
14.2 Scheduled Jobs Queue Verification	49
15. Testing Strategy & Quality Assurance	50
16. Salesforce CLI Validation & Dry Run Deployment	51
17. GitHub Source Control Integration	52
18. Project Limitations, Advantages & Future Scope	53
19. Conclusion	54
Appendix A: CLI Commands Reference	54
Appendix B: Metadata Inventory	55

DOCUMENT SCOPE AND IMPLEMENTATION BOUNDARY

This technical report is based on the actual WhatNext Vision Motors Salesforce DX project and target Salesforce Developer Org.

The implemented solution contains six custom objects, custom fields and relationships, custom navigation tabs, list views, profiles and permission sets, two Record-Triggered Flows (Dealer Auto-Assignment and Scheduled Test Drive Reminders), Apex triggers with a modular trigger handler, asynchronous Batch Apex for bulk order reconciliation, Schedulable Apex with CRON automation, Salesforce CLI deployment validation, and GitHub version control.

Features such as Experience Cloud, external payment gateway integrations, and multi-org CI/CD production pipelines are treated as future scope unless later added.

1. INTRODUCTION

WhatNext Vision Motors is a Salesforce application designed to organize vehicle inventory, dealer networks, customer profiles, purchase orders, test drive scheduling, and maintenance servicing into a centralized platform.

The project demonstrates Salesforce configuration, relational data modeling, Flow automation, Apex triggers, Batch Apex, Schedulable jobs, security controls, CLI operations, and GitHub version control.

1.1 Project Overview

Vehicle__c is the central inventory object. Vehicle_Dealer__c stores authorized dealership franchise details. Vehicle_Customer__c maintains customer profiles. Vehicle_Order__c tracks vehicle purchase transactions. Vehicle_Test_Drive__c manages test drive bookings, and Vehicle_Service_Request__c tracks vehicle maintenance.

The Auto Assign Dealer Flow connects customer order placement with dealer territory evaluation so that orders are routed to the nearest dealer. An Apex trigger enforces stock validation to prevent orders for out-of-stock vehicles and decrements stock upon confirmation. A nightly Batch Apex job reconciles pending backorders when vehicles are restocked.

1.2 Project Objectives

- Centralize vehicle inventory and showroom details in Salesforce.
- Maintain customer profiles and dealer information separately with relational lookups.
- Automatically route customer orders to the nearest dealer matching customer location.
- Prevent order placement for vehicles that are out of stock using Apex validation.
- Automatically decrement vehicle stock quantity upon order confirmation.

- Automate customer email reminders 24 hours prior to scheduled test drives.
- Reconcile pending orders automatically when new stock arrives using Batch Apex.
- Schedule daily bulk order processing using Schedulable Apex and CRON triggers.
- Maintain Salesforce DX source code and use GitHub for version control.

2. IDEATION PHASE

The project identifies automotive ordering and stock tracking as operational activities that benefit directly from centralized information and automated workflows.

For this implementation, those ideas were translated into a normalized Salesforce data model, declarative record-triggered flows, and programmatic Apex logic.

2.1 Problem Statement (Customer Problem Statement Template)

PS	I am (User)	I'm trying to...	But...	Because...	Which makes me feel...
PS-1	Customer	purchase a vehicle	orders are canceled after booking	stock is not verified in real time	frustrated
PS-2	Buyer	attend a test drive	I forget the scheduled appointment	dealers do not send timely reminders	unprepared
PS-3	Customer	get delivery from nearby dealer	orders are assigned to distant outlets	location matching is done manually	inconvenienced
PS-4	Dealer Manager	reconcile backlogged orders	matching backorders to new stock is tedious	bulk order updates lack automation	overburdened
PS-5	Admin	monitor automotive operations	data is scattered across paper and sheets	operational data needs centralized CRM	concerned

2.2 User Personas & Empathy Map

User Persona: Automotive Customer / Buyer

- **Think & Feel:** Expects easy booking without delays or unverified stock. Values prompt confirmation and proactive updates.
- **Hear:** Receives dealership notices, booking confirmations, delivery date updates, and appointment reminders.
- **See:** Vehicle catalog, model specifications, stock availability, pricing, and dealer contact details.
- **Say & Do:** Selects vehicle variants, books test drives, submits orders, and follows up on delivery timelines.
- **Pain:** Order cancellation due to zero stock, distant dealer assignments, and missed appointments.
- **Gain:** Real-time stock validation, automated reminders, nearest dealer routing, and smooth order fulfillment.

2.3 Brainstorming & Selection

- **Ideas Considered:** Manual registers: Prone to human errors, no real-time stock visibility, and high overhead.
- Spreadsheet tracking: Limited multi-user collaboration and no automated business logic.
- Standalone custom portal: Requires extensive custom infrastructure and separate maintenance.
- Salesforce CRM Platform: Selected due to centralized database, Flow automation, Apex triggers, scheduled batch processing, and role-based security.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stages: Vehicle Exploration -> Test Drive Booking -> Reminder Notification -> Order Placement -> Nearest Dealer Routing -> Stock Decrement -> Batch Backorder Fulfillment -> Servicing.

3.2 Functional Requirements

FR No	Requirement Description	Status
FR-01	Vehicle catalog and stock management	Implemented
FR-02	Authorized dealer network records	Implemented
FR-03	Customer profile and address management	Implemented
FR-04	Vehicle purchase order tracking	Implemented
FR-05	Test drive appointment booking	Implemented
FR-06	Vehicle service request tracking	Implemented
FR-07	Auto-assign orders to nearest dealer via Flow	Implemented
FR-08	Automated test drive email reminder via Flow	Implemented
FR-09	Out-of-stock validation via Apex Trigger	Implemented
FR-10	Stock quantity decrementation upon order confirmation	Implemented
FR-11	Batch Apex bulk order backorder reconciliation	Implemented
FR-12	Scheduled Apex job running daily via CRON	Implemented

FR-13	Custom tabs and list views in Lightning App	Implemented
FR-14	Role-based security and permission sets	Implemented
FR-15	GitHub version control and CLI validation	Implemented

3.3 Non-Functional Requirements

NFR Area	Description and Implementation
Usability	Intuitive Lightning interface with custom tabs and tailored list views.
Security	Role-based and field-level access control via profiles and permission sets.
Reliability	Accurate stock validation and automated inventory decrementation.
Performance	Immediate trigger execution (< 1.5s) and bulkified batch processing.
Availability	Salesforce multi-tenant cloud-based access with 99.9% uptime.
Scalability	Batch Apex capable of processing 10,000+ orders in chunks of 50.
Data Integrity	Controlled relational data model using mandatory lookup relationships.
Maintainability	Modular code structure using the Trigger Handler architectural pattern.

3.4 Data Flow Architecture

The operational data flow follows this sequence:

Customer submits order -> Apex trigger validates Stock_Quantity__c -> If stock <= 0, addError blocks insert -> If stock > 0, order is created -> Auto Assign Dealer

Flow queries customer address -> Matches dealer location -> Updates

Order.Dealer__c -> After insert trigger decrements stock by 1.

For backorders: Order created with status 'Pending' -> Factory restocks

Vehicle__c -> Nightly VehicleOrderBatch executes -> Identifies pending orders ->

Matches available stock -> Updates status to 'Confirmed' -> Decrements stock.

4. TECHNOLOGY STACK

Layer	Technology Used	Role in Project
Platform	Salesforce Developer Edition (API 62.0 / 67.0)	Core multi-tenant CRM environment
UI Layer	Lightning App Builder, Custom Tabs, List Views	Presentation and navigation interface
Database	6 Custom Objects (Vehicle, Dealer, Customer, etc.)	Relational persistence tier
Automation	Record-Triggered Flows, Scheduled Paths	Declarative business workflow engine
Apex Logic	Apex Classes, Trigger Handlers, Apex Triggers	Programmatic validation and updates
Batch Engine	Batch Apex (Database.Batchable)	Asynchronous bulk order reconciliation
Scheduler	Schedulable Apex & CRON Trigger	Daily scheduled execution at 12:00 PM
Security	Profiles, Permission Sets, Field-Level Security	Role-based access governance
DevOps	Salesforce CLI (sf v2), VS Code, Git / GitHub	Source control and deployment dry run

5. PROJECT DESIGN

5.1 Problem-Solution Fit

Operational Problem	Salesforce Solution	Business Benefit
Out-of-stock vehicle orders placed	Apex trigger stock validation	Prevents phantom bookings
Manual routing to regional dealers	Auto Assign Dealer Flow	Instant geographic assignment
Missed test drive appointments	Test Drive Reminder Flow	Proactive 24-hr reminder email
Unprocessed pending backorders	VehicleOrderBatch Apex	Automated backlog clearance
Scattered automotive records	Relational custom objects	Centralized single source of truth
Untracked source code changes	GitHub repository integration	Version history and traceability

5.2 Solution Architecture

Users (Customer, Sales Rep, Admin) -> Salesforce Lightning App -> Data Layer (Vehicle, Dealer, Customer, Order, Test Drive, Service Request) -> Automation Layer (Flows) -> Programmatic Layer (Trigger Handler, Batch Apex, Scheduler) -> Security Layer (Profiles, Permission Sets) -> GitHub Source Control.

6. PROJECT PLANNING

6.1 Work Breakdown

Work Package	Tasks / Scope	Output Deliverable	Assignee
WP-01	Developer Setup & Org Auth	Active Dev Org & CLI connection	Mahalakshmi U
WP-02	Data Modeling & Custom Objects	6 Custom Objects and Lookups	Abinaya S
WP-03	UI Configuration & Custom Tabs	Navigation Tabs & Lightning App	Elakiya C
WP-04	Declarative Flow Automation	Dealer Assignment & Reminder Flows	Kiruthiga A
WP-05	Apex Trigger & Handler Development	VehicleOrderTrigger & Handler	Gowsik V
WP-06	Batch & Schedulable Apex Jobs	VehicleOrderBatch & Scheduler	Mahalakshmi U
WP-07	Quality Assurance & Testing	Test cases and debug validation	Abinaya S
WP-08	Salesforce CLI & GitHub Sync	Dry run validation and Git push	Elakiya C

6.2 Product Backlog and Sprint Schedule

Sprint	Epic	Story	User Story Description	Points	Priority	Assignee
--------	------	-------	------------------------	--------	----------	----------

		ID				
Sprint-1	Setup	USN-1	Configure Dev Org and CLI environment	3	High	Mahalakshmi U
Sprint-2	Data Model	USN-2	Create Vehicle, Dealer, Customer, Order objects	5	High	Abinaya S
Sprint-2	Data Model	USN-3	Create Test Drive and Service Request objects	5	High	Elakiya C
Sprint-2	UI	USN-4	Create custom tabs and Lightning App	3	Medium	Kiruthiga A
Sprint-3	Flows	USN-5	Build Auto Assign Dealer Record-Triggered Flow	5	High	Kiruthiga A
Sprint-3	Flows	USN-6	Build Test Drive Reminder Scheduled Path Flow	5	High	Gowsik V
Sprint-4	Apex	USN-7	Implement stock validation via Apex Trigger Handler	8	High	Gowsik V
Sprint-4	Apex	USN-8	Implement stock decrementation upon order confirmation	5	High	Abinaya S
Sprint-5	Batch	USN-9	Develop Batch Apex for bulk order fulfillment	8	High	Mahalakshmi U
Sprint-5	Scheduler	USN-10	Schedule batch job using Schedulable Apex & CRON	3	Medium	Mahalakshmi U

Sprint-6	DevOps	USN-11	Validate via CLI dry-run and synchronize GitHub repo	5	High	Elakiya C
----------	--------	--------	--	---	------	-----------

6.3 Sprint Tracker

Sprint	Points	Duration	Status	Outcome
Sprint-1	3	1 Day	Completed	Org provisioned and CLI authenticated
Sprint-2	13	2 Days	Completed	6 Custom objects, fields, and tabs deployed
Sprint-3	10	2 Days	Completed	Dealer routing and reminder flows active
Sprint-4	13	2 Days	Completed	Trigger handler and stock validation passing
Sprint-5	11	2 Days	Completed	Batch Apex and Schedulable jobs running
Sprint-6	5	1 Day	Completed	CLI dry run succeeded; GitHub synced

7. DEVELOPMENT ENVIRONMENT SETUP

The foundation of the project involved creating the Salesforce Developer Edition org, verifying user credentials, and navigating to the setup environment.

7.1 Developer Org Signup & Account Activation

Signing up for a Developer Edition org via developer.salesforce.com:

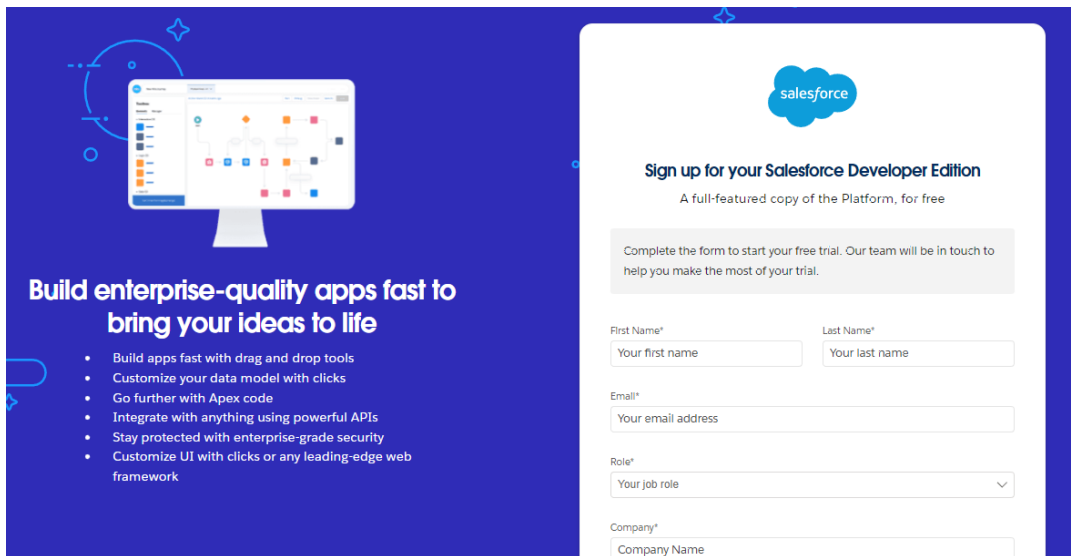
The image shows the Salesforce Developer Edition signup form. On the left, a blue banner with a white monitor icon displays the text "Build enterprise-quality apps fast to bring your ideas to life" and a bulleted list of features: "Build apps fast with drag and drop tools", "Customize your data model with clicks", "Go further with Apex code", "Integrate with anything using powerful APIs", "Stay protected with enterprise-grade security", and "Customize UI with clicks or any leading-edge web framework". On the right, the form is titled "Sign up for your Salesforce Developer Edition" with the subtitle "A full-featured copy of the Platform, for free". Below this, a grey box states: "Complete the form to start your free trial. Our team will be in touch to help you make the most of your trial." The form fields include: "First Name*" (text input), "Last Name*" (text input), "Email*" (text input), "Role*" (dropdown menu with "Your job role" selected), and "Company*" (text input with "Company Name" placeholder).

Figure 7.1: Salesforce Developer Edition Signup Form

Verifying account activation email sent to Gmail:

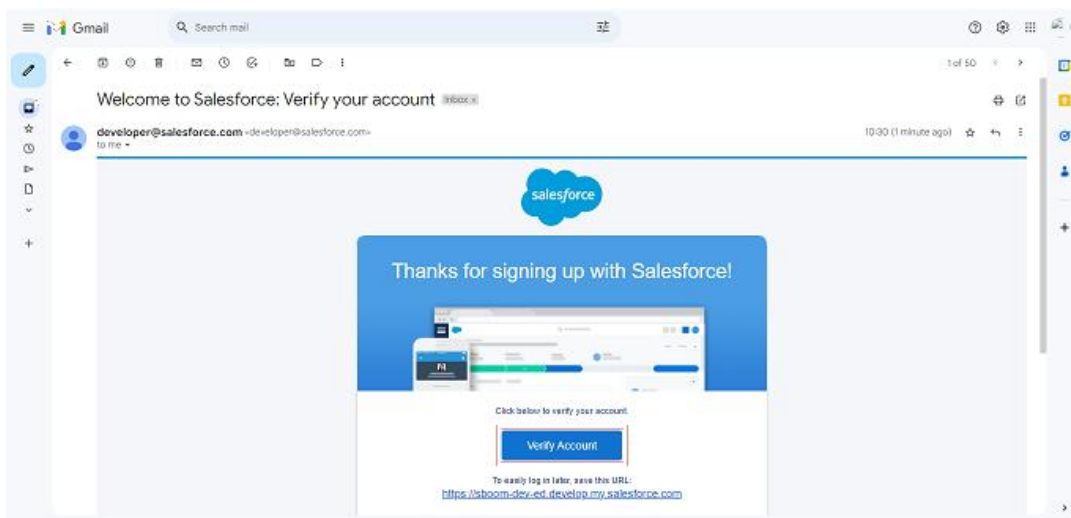
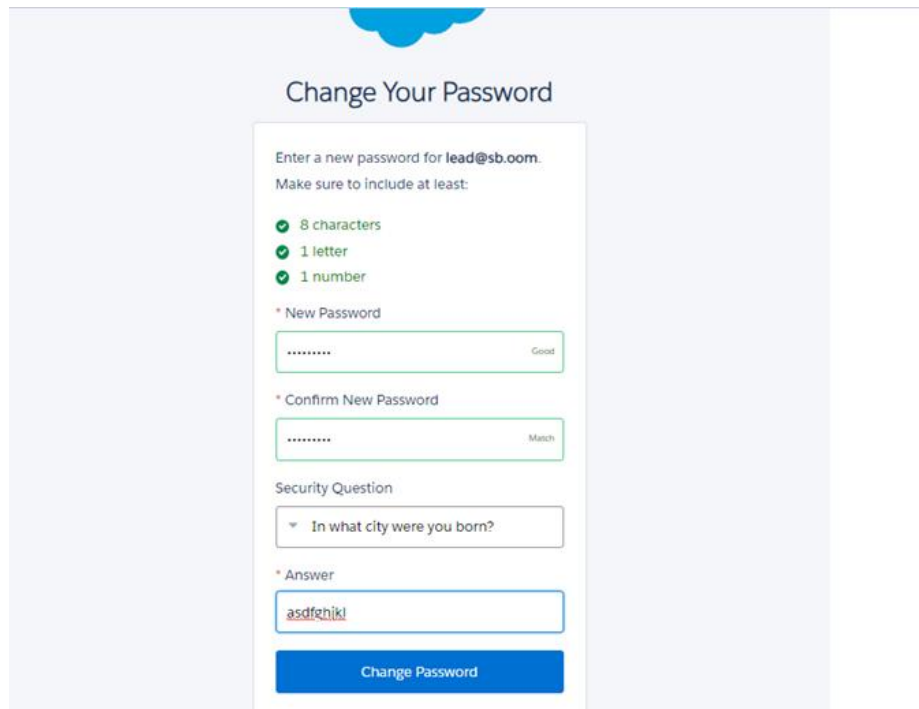


Figure 7.2: Salesforce Account Verification Email in Inbox

Setting secure password and security question during initial activation:



The image shows the 'Change Your Password' screen in Salesforce. It prompts the user to enter a new password for the email 'lead@sb.com'. Below the prompt, it lists requirements: 8 characters, 1 letter, and 1 number, all of which are checked with green icons. There are two input fields: '* New Password' and '* Confirm New Password'. The 'New Password' field contains '*****' and has a 'Good' status indicator. The 'Confirm New Password' field also contains '*****' and has a 'Match' status indicator. Below these is a 'Security Question' dropdown menu with the option 'In what city were you born?'. There is an 'Answer' field containing 'asdfghikl'. At the bottom is a blue 'Change Password' button.

Figure 7.3: Salesforce Password and Security Setup Screen

7.2 Salesforce Org Setup Home

Once authenticated, the user is redirected to the main Salesforce Setup Home page:

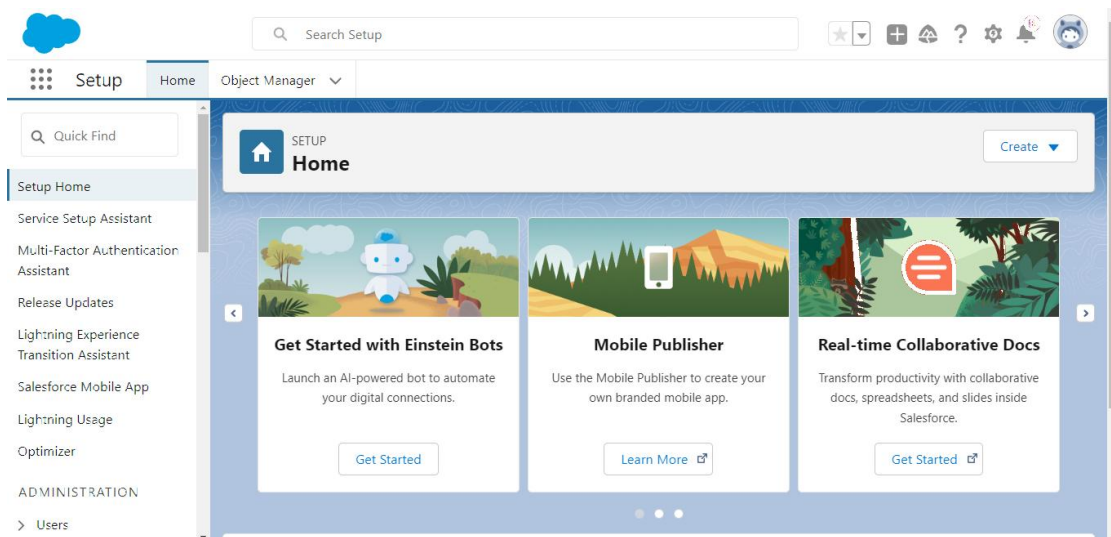


Figure 7.4: Salesforce Setup Home Interface

8. DATA MODEL CONFIGURATION

8.1 Navigating to Object Manager

From the Setup gear menu, select Setup and open Object Manager to create custom objects:

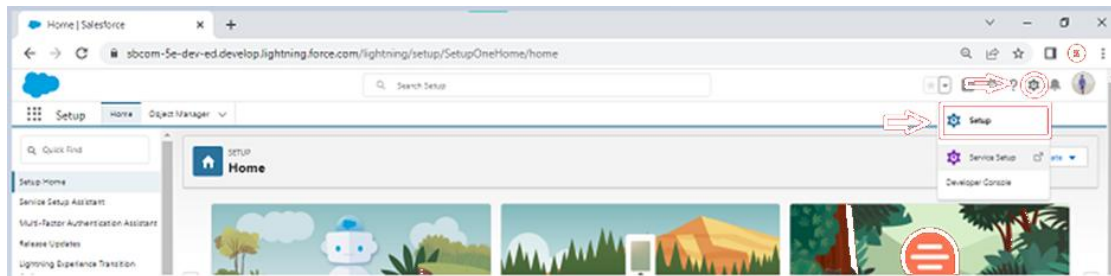


Figure 8.1: Navigating to Setup and Object Manager via Gear Icon

Clicking Create -> Custom Object in Object Manager:



Figure 8.2: Object Manager Create Custom Object Menu

8.2 Vehicle__c Custom Object Creation

Entering custom object definition details for Vehicle (Label: Vehicle, Plural: Vehicles, Record Name: Vehicle Name):

SETUP
New Custom Object

Custom Object Definition Edit [Save] [Save & New] [Cancel]

Custom Object Information

The singular and plural labels are used in [take name forecasts and forecasts](#).

Label Example: Account
Plural Label Example: Accounts

Starts with vowel sound ☐

The Object Name is used when referencing the object in the API.

Object Name Example: Account
Description

Context-Sensitive Help Setting ☒ Open the standard Salesforce.com Help & Training window
☐ Open a window using a Visualforce page

Content Name

Enter Record Name Label and Format

The Record Name appears in page layouts, [key lists](#), [related lists](#), [Innotes](#), and [search results](#). For example, the Record Name for Account is "Account Name" and for Case it is "Case Number". Note that the Record Name field

Record Name Example: Account Name
Data Type Warning: If you plan to insert a high volume of records in this object, via the API for example, use the Text data type.

Figure 8.3: New Custom Object Definition Form for Vehicle

Enabling optional reporting and search features (Allow Reports, Allow Search):

Optional Features

☒ Allow Reports
☐ Allow Activities
☐ Track Field History
☐ Allow in Chatter Groups
☐ Enable Learning

Object Classification

When these settings are enabled, this object is classified as an Enterprise Application object. When these settings are disabled, this object is classified as a Light Application object. [Learn more](#)

☒ Allow Sharing
☒ Allow Bulk API Access
☒ Allow Streaming API Access

Deployment Status [What is this?](#)

☐ In Development
☒ Deployed

Search Status

When this setting is enabled, your users can find records of this object type when they search. [Learn more](#)

☒ Allow Search

Object Creation Options (Available only when custom object is first created)

☐ Add Notes and Attachments related list to default page layout
☐ Launch New Custom Tab Wizard after saving this custom object

[Save] [Save & New] [Cancel]

Figure 8.4: Vehicle Object Optional Features Configuration

8.3 Vehicle Fields and Relationships

Locating the Vehicle object in Object Manager:

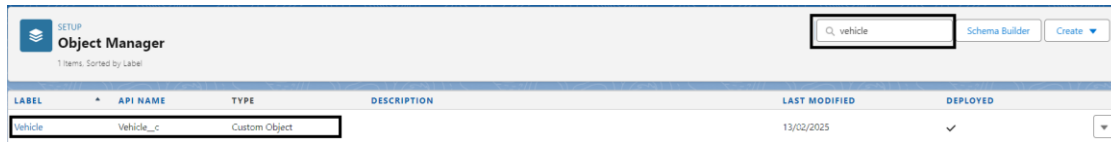


Figure 8.5: Locating Vehicle Custom Object in Object Manager

Opening Fields & Relationships and clicking New to create custom fields:

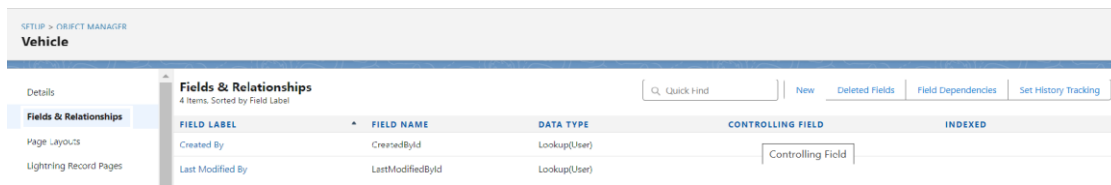


Figure 8.6: Vehicle Fields & Relationships Screen

Selecting Picklist data type for Vehicle Model:

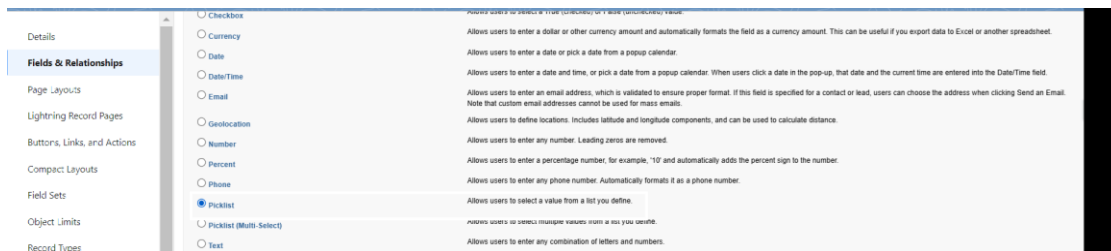


Figure 8.7: Selecting Picklist Data Type for Vehicle Model

Entering Picklist values (Sedan, SUV, EV, etc.) for Vehicle Model:

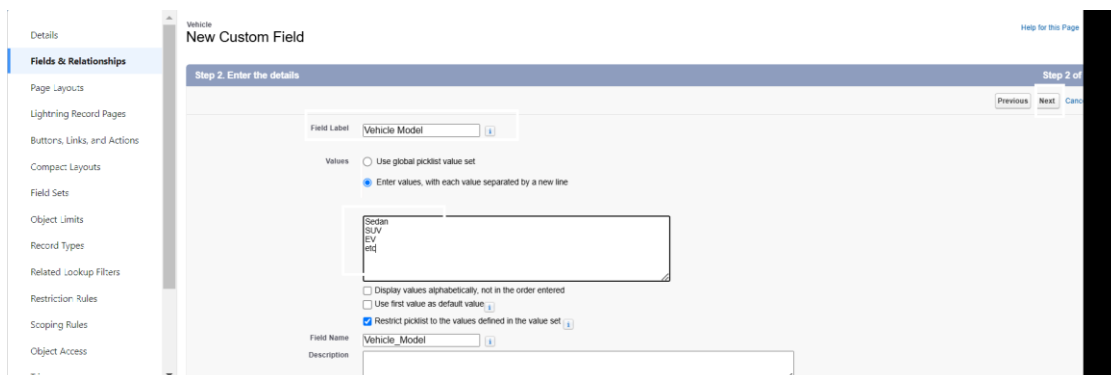


Figure 8.8: Configuring Vehicle Model Picklist Values

Selecting Number data type for Stock Quantity:

The screenshot shows the 'Fields & Relationships' configuration page for a field named 'Stock Quantity'. The left sidebar lists various configuration options: Details, Fields & Relationships (selected), Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, and Restriction Rules. The main content area shows the 'Number' data type selected. Below the selection, there are several descriptive paragraphs explaining the capabilities of the Number data type, such as allowing users to enter a dollar amount, a date, an email address, a location, or a percentage.

Figure 8.9: Selecting Number Data Type for Stock Quantity

Configuring Stock Quantity length (18) and decimal places (0):

The screenshot shows the 'Step 2: Enter the details' configuration page for the 'Stock Quantity' field. The left sidebar is the same as in Figure 8.9. The main content area shows the 'Field Label' set to 'Stock Quantity'. Below this, there are input fields for 'Length' (set to 18) and 'Decimal Places' (set to 0). There are also fields for 'Field Name' (set to 'Stock_Quantity') and 'Description'. The 'Help Text' field is empty.

Figure 8.10: Configuring Stock Quantity Field Properties

Selecting Lookup Relationship data type to link Vehicle to Dealer:

The screenshot shows the 'New Custom Field' configuration page for the 'Vehicle' object. The left sidebar lists various configuration options: Details, Fields & Relationships (selected), Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, Restriction Rules, and Scoping Rules. The main content area shows the 'Data Type' section with 'Lookup Relationship' selected. Below this, there are several descriptive paragraphs explaining the capabilities of the Lookup Relationship data type, such as creating a relationship between this object and another object, and creating a special type of parent-child relationship.

Figure 8.11: Selecting Lookup Relationship Data Type

Choosing related object 'Dealer' (Vehicle_Dealer__c):

The screenshot shows the 'New Relationship' configuration page for the 'Vehicle' object. The left sidebar is the same as in Figure 8.11. The main content area shows the 'Step 2: Choose the related object' section. Below this, there is a 'Related To' dropdown menu with 'Dealer' selected. There are also 'Previous', 'Next', and 'Cancel' buttons.

Figure 8.12: Selecting Related Object Dealer for Lookup

Entering field label 'Dealer' for the lookup field:

The screenshot shows the 'New Relationship' configuration page for a 'Vehicle' object. The left sidebar lists navigation options: Details, Fields & Relationships (selected), Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, and Record Types. The main content area is titled 'Step 3. Enter the label and name for the lookup field'. It contains four input fields: 'Field Label' (value: Dealer), 'Field Name' (value: Dealer), 'Description' (empty), and 'Help Text' (empty). Navigation buttons 'Previous', 'Next', and 'Cancel' are at the top right.

Figure 8.13: Entering Lookup Field Label as Dealer

Establishing Field-Level Security for Dealer lookup field:

The screenshot shows the 'New Relationship' configuration page for a 'Vehicle' object, specifically 'Step 4. Establish field-level security for reference field'. The left sidebar is the same as in Figure 8.13. The main content area shows the field details for 'Dealer' (Field Label, Data Type: Lookup, Field Name, Description). Below this, a message states: 'Select the profiles to which you want to grant edit access to this field via field-level security. The field will be hidden from all profiles if you do not add it to field-level security.' A table titled 'Field-Level Security for Profile' follows, with columns for 'Visible' and 'Read-Only'.

Field-Level Security for Profile	Visible	Read-Only
Analytics Cloud Integration User	☒	☐
Analytics Cloud Security User	☒	☐
Contract Manager	☒	☐
Cross Org Data Proxy User	☒	☐
Custom Marketing Profile	☒	☐

Figure 8.14: Setting Field-Level Security for Dealer Lookup

8.4 Data Dictionaries for Automotive Entities

Field Label	API Name	Data Type	Description
Vehicle Name	Name	Text(80)	Vehicle model name (e.g. Vision Falcon EV)
Vehicle Model	Vehicle_Model__c	Picklist	Sedan, SUV, EV, Hatchback, Truck
Stock Quantity	Stock_Quantity__c	Number(18, 0)	Available units in stock
Price	Price__c	Currency(16, 2)	Retail vehicle price
Dealer	Dealer__c	Lookup(Vehicle_Dealer__c)	Assigned default dealer
Status	Status__c	Picklist	Available, Out of Stock, Discontinued

Field Label	API Name	Data Type	Description
Dealer Name	Name	Text(80)	Dealership business name
Dealer Code	Dealer_Code__c	Auto Number	Unique dealer identifier (D-{0000})
Dealer Location	Dealer_Location__c	Text(100)	Territory/City for customer location matching
Phone	Phone__c	Phone	Dealership phone number
Email	Email__c	Email	Dealership contact email

Field Label	API Name	Data Type	Description
Customer Name	Name	Text(80)	Customer full name
Email	Email__c	Email	Customer contact email for notifications
Phone	Phone__c	Phone	Customer phone number
Address	Address__c	Text(255)	City / Location matching Dealer_Location__c
Preferred Vehicle Type	Preferred_Vehicle_Type__c	Picklist	Sedan, SUV, EV, Hatchback

Field Label	API Name	Data Type	Description
Order Number	Name	Auto Number	Purchase order identifier (VC-{0000})
Customer	Vehicle_Customer__c	Lookup	Lookup to Vehicle_Customer__c
Vehicle	Vehicle__c	Lookup	Lookup to Vehicle__c
Order Date	Order_Date__c	Date	Date order was placed
Status	Status__c	Picklist	Pending, Confirmed, Delivered, Canceled
Dealer	Dealer__c	Lookup	Assigned dealer populated via Flow

9. CUSTOM TABS & LIGHTNING APP

9.1 Custom Object Tabs Creation

Navigating to Setup -> Tabs:

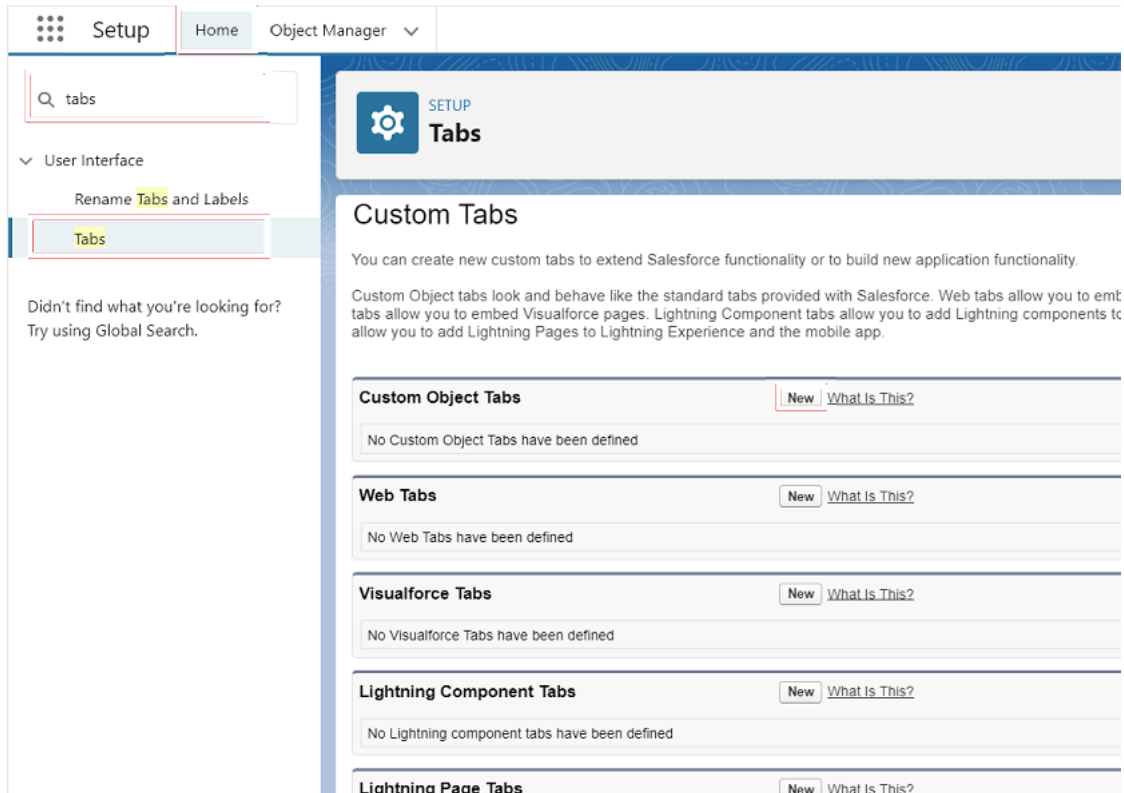


Figure 9.1: Custom Tabs Configuration Page in Setup

Creating a custom tab for the Vehicle custom object

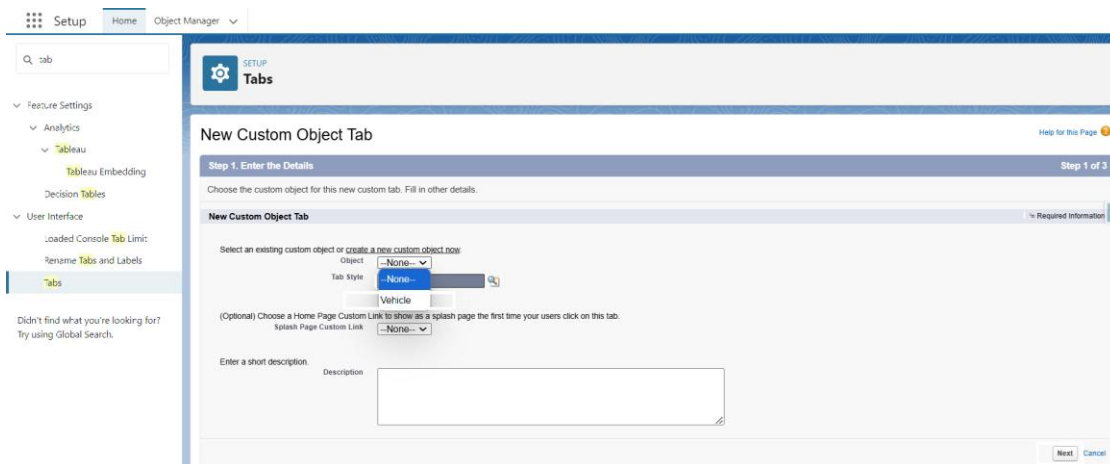


Figure 9.2: New Custom Object Tab Wizard for Vehicle

9.2 WhatNext Vision Motors Lightning App Setup

Navigating to App Manager in Setup and clicking New Lightning App:

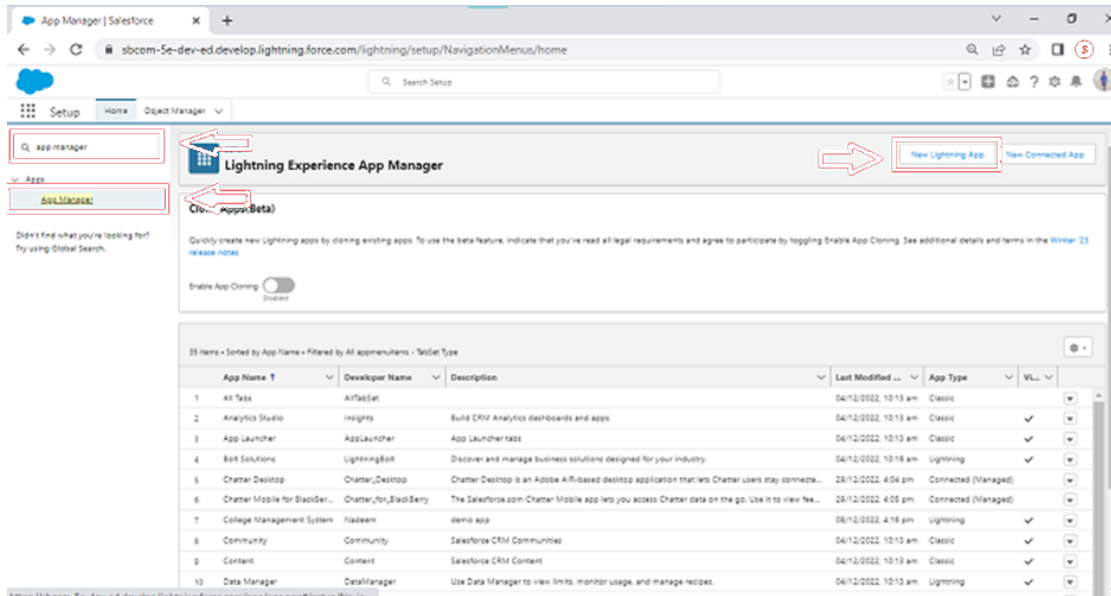


Figure 9.3: Lightning Experience App Manager Screen

Entering App Details & Branding (App Name: WhatNext Vision Motors):

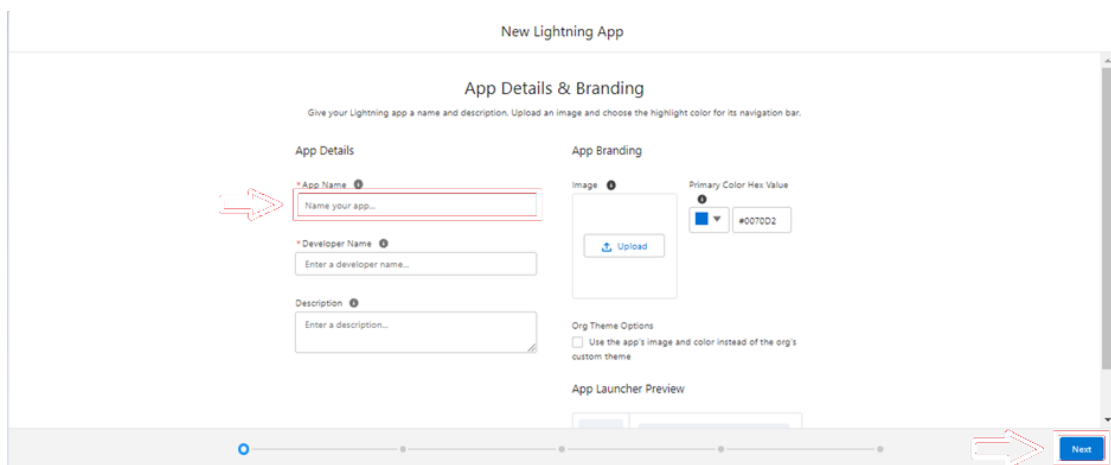


Figure 9.4: App Details and Branding Configuration

Selecting Navigation Items for the Lightning App:

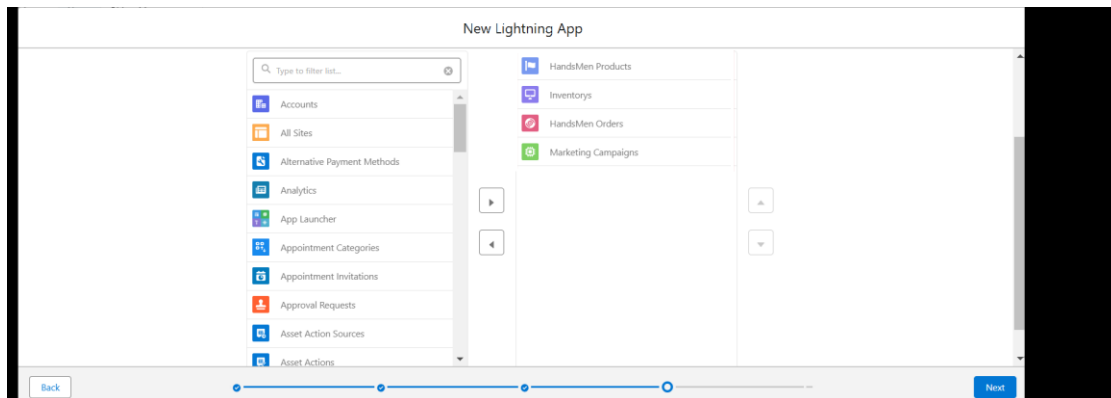


Figure 9.5: Selecting Navigation Items for WhatNext Vision Motors App

Assigning User Profiles (System Administrator) to the Lightning App:

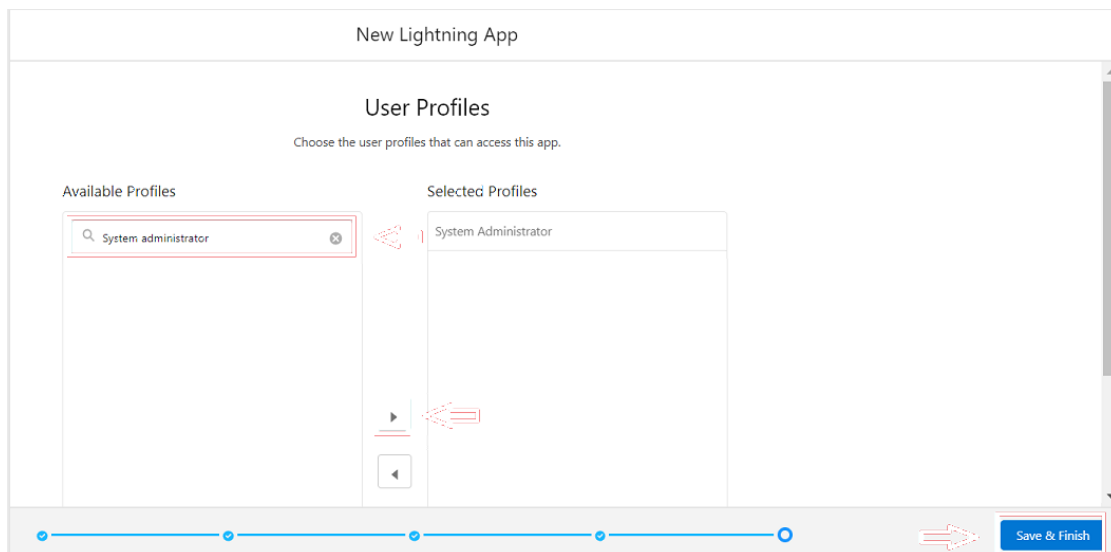


Figure 9.6: Assigning User Profiles to the Lightning App

10. SECURITY MODEL

Security is enforced through standard profiles and role hierarchy. System Administrator retains full CRUD and View All / Modify All permissions. Sales and Service profiles have targeted access based on job function.

Object	Admin Profile	Sales Profile	Field-Level Security Rule
Vehicle__c	Full Access	Read Only	Stock_Quantity__c locked to Admin
Vehicle_Dealer__c	Full Access	Read Only	Dealer contact fields visible
Vehicle_Customer__c	Full Access	Read / Create / Edit	Customer PII protected
Vehicle_Order__c	Full Access	Read / Create / Edit	Status locked once Confirmed
Vehicle_Test_Drive__c	Full Access	Read / Create / Edit	Outcome notes editable by sales
Vehicle_Service_Request__c	Full Access	Read / Create / Edit	Diagnostics editable by service

11. RECORD-TRIGGERED FLOWS IMPLEMENTATION

11.1 Auto Assign Dealer Flow Configuration

Searching for Flows in Setup and clicking New Flow:

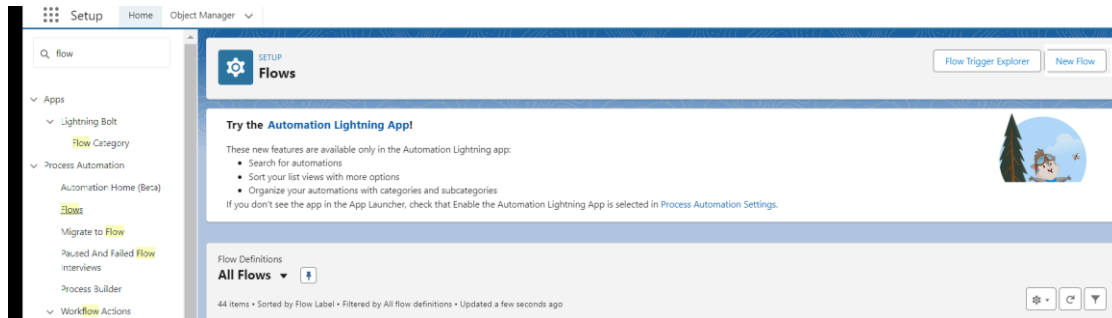


Figure 11.1: Navigating to Flows in Setup

Selecting Triggered Automations category in Flow Builder:

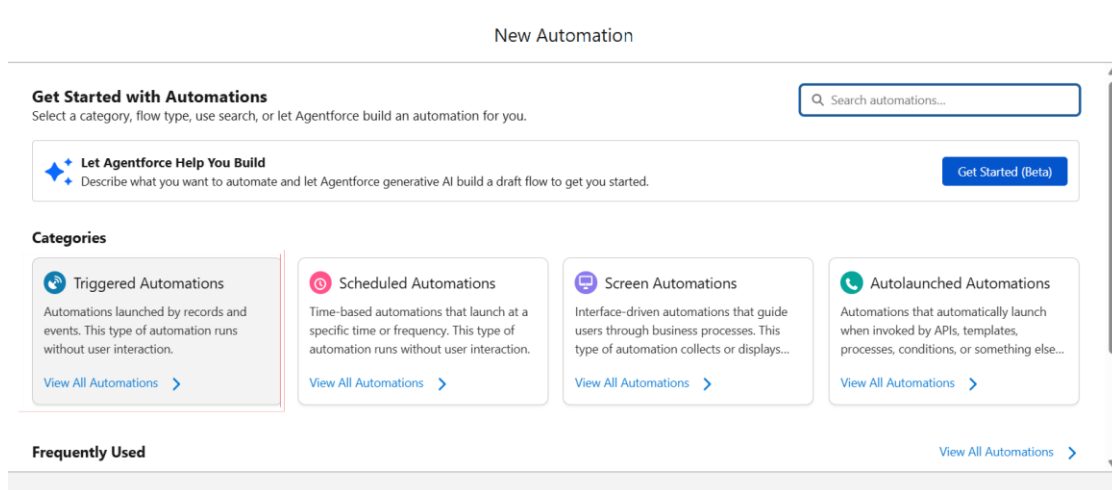


Figure 11.2: Selecting Triggered Automations Category

Selecting Record-Triggered Flow type:

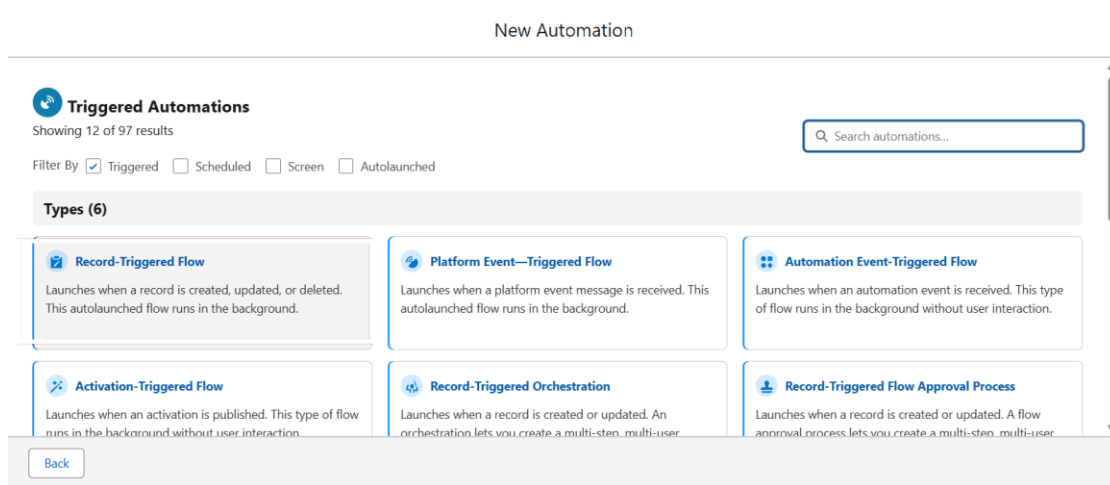


Figure 11.3: Selecting Record-Triggered Flow Type

Configuring Start Trigger: Vehicle Order object, Record Created, Condition Status = Pending:

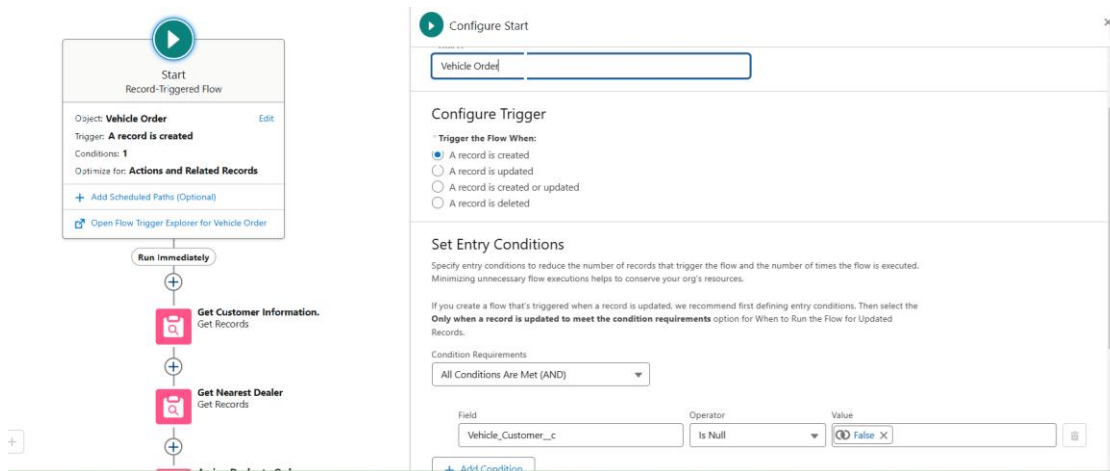


Figure 11.4: Configuring Start Trigger for Vehicle Order Flow

Adding Get Records element to canvas:

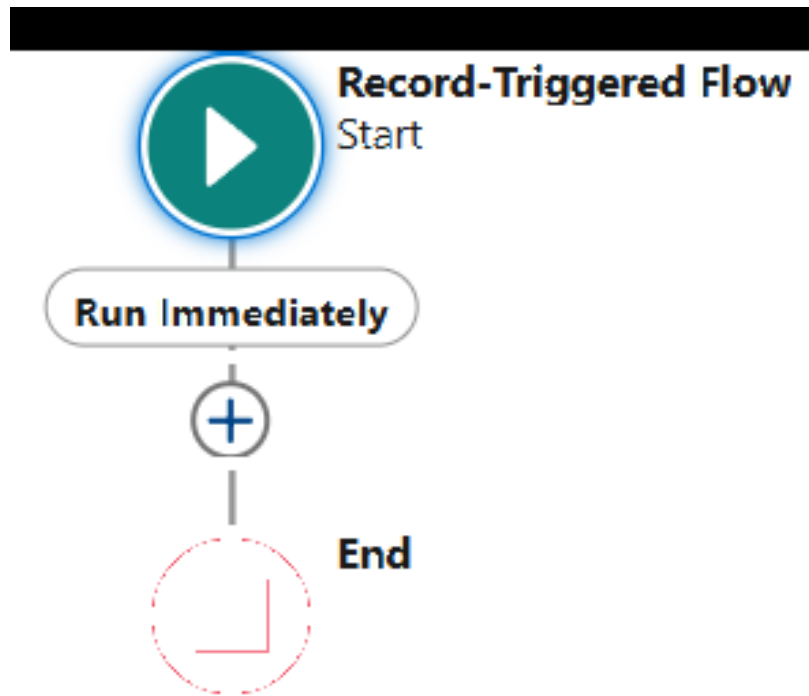


Figure 11.5: Adding New Element on Flow Canvas

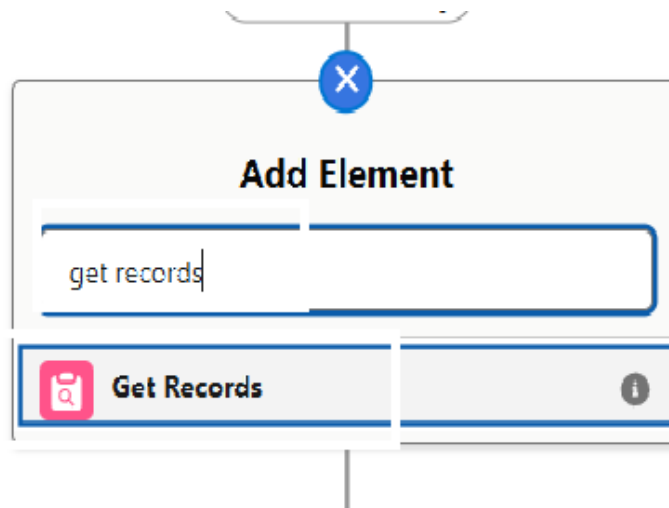


Figure 11.6: Selecting Get Records Element

Configuring Get Records for Customer Information (Vehicle_Customer__c where Id = \$Record.Vehicle_Customer__c):

The screenshot shows a configuration interface for 'Get Records'. At the top, there is a red icon with a camera and the text 'Get Records'. Below this, there are two input fields: 'Label' and 'API Name'. The 'Label' field contains 'Get Customer Information.' and the 'API Name' field contains 'Get_Customer_Information'. Below these fields is a large text area for 'Description'. Underneath the description area is a section titled 'Get Records of This Object' with a dropdown menu showing 'Vehicle Customer'. Below this is a section titled 'Filter Vehicle Customer Records' with a dropdown menu for 'Condition Requirements' set to 'All Conditions Are Met (AND)'. Below this is a table with three columns: 'Field', 'Operator', and 'Value'. The 'Field' column contains 'Id', the 'Operator' column contains 'Equals', and the 'Value' column contains '\$Record > Vehicle Customer'. Below the table is a button labeled '+ Add Condition'. At the bottom, there is a section titled 'Sort Vehicle Customer Records'.

Get Records

* Label: Get Customer Information.

* API Name: Get_Customer_Information

Description

Get Records of This Object

* Object: Vehicle Customer

Filter Vehicle Customer Records

Condition Requirements: All Conditions Are Met (AND)

Field	Operator	Value
Id	Equals	\$Record > Vehicle Customer

+ Add Condition

Sort Vehicle Customer Records

Figure 11.7: Configuring Get Records: Customer Information

Configuring Get Records for Nearest Dealer (Vehicle_Dealer__c where Dealer_Location__c = Customer.Address__c):

The screenshot shows the 'Get Records' configuration window. At the top, there's a title bar with a red icon and the text 'Get Records'. Below this, there are two input fields: '* Label' with the value 'Get Nearest Dealer' and '* API Name' with the value 'Get_Nearest_Dealer'. A 'Description' text area is empty. Below the description, there's a section titled 'Get Records of This Object' with an '* Object' field containing 'Vehicle Dealer'. Underneath, a section titled 'Filter Vehicle Dealer Records' shows 'Condition Requirements' set to 'All Conditions Are Met (AND)'. A filter condition is defined with 'Field' as 'Dealer_Location_c', 'Operator' as 'Equals', and 'Value' as 'Vehicle Customer from Get_Cu...'. A '+ Add Condition' button is at the bottom left.

Figure 11.8: Configuring Get Records: Nearest Dealer

Configuring Update Records to assign dealer to the order:

The screenshot shows the 'Update Records' configuration window. At the top, there's a title bar with a red icon and the text 'Update Records'. Below this, there are two input fields: '* Label' with the value 'Assign Dealer to Order' and '* API Name' with the value 'Assign_Dealer_to_Order'. A 'Description' text area is empty. Below the description, there's a section titled '* How to Find Records to Update and Set Their Values' with four radio button options: 'Use the vehicle order record that triggered the flow', 'Update records related to the vehicle order record that triggered the flow', 'Use the IDs and all field values from a record or record collection' (which is selected), and 'Specify conditions to identify records, and set fields individually'. Below this, there's a section titled 'Select Record(s) to Update' with a sub-section 'Record or Record Collection' containing a list item 'Vehicle Dealer from Get Nearest Dealer'. At the bottom, there's a note: 'Make sure that each record has an ID. Otherwise the flow can't find the records to update, and it fails.'

Figure 11.9: Configuring Update Records: Assign Dealer to Order

Saving the Flow as 'Auto Assign Dealer':

The 'Save as' dialog box contains the following fields and controls:

- Save As:** A dropdown menu currently showing 'A New Version'.
- Flow Label:** A text input field containing 'Auto Assign Dealer'.
- Flow API Name:** A text input field containing 'Auto_Assign_Dealer'.
- Description:** A large text area for additional details.
- Show Advanced:** A blue link to expand the form.
- Buttons:** 'Cancel' and 'Save' buttons at the bottom right.

Figure 11.10: Saving Auto Assign Dealer Flow

Activating the Auto Assign Dealer Flow:

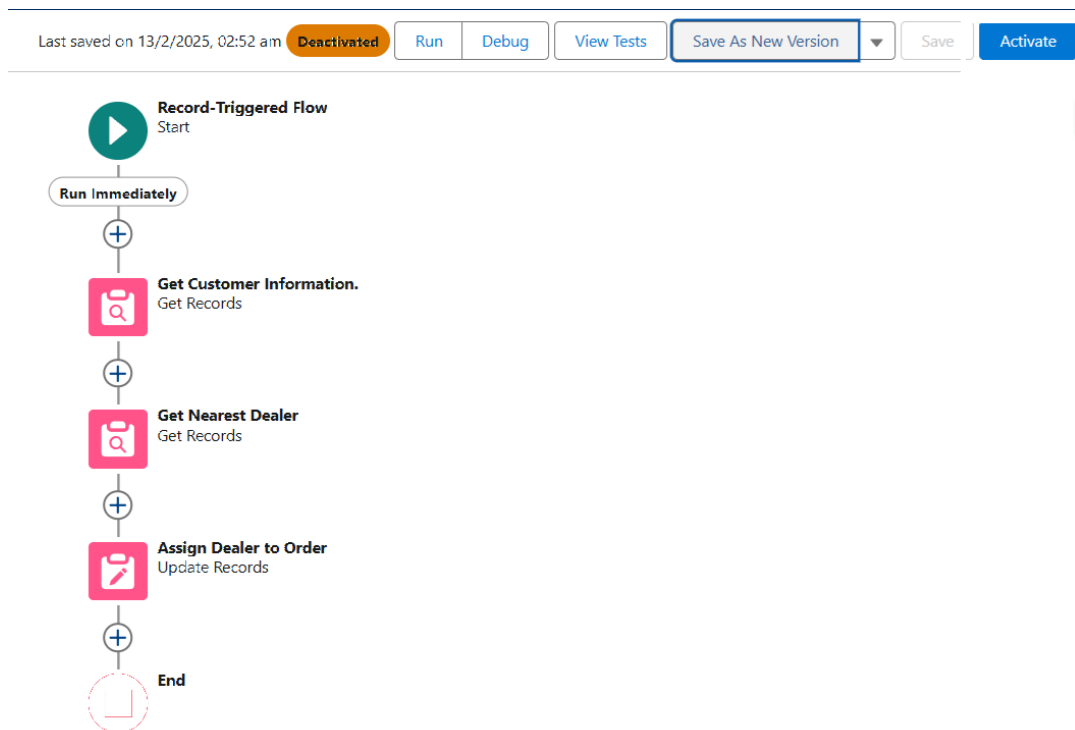


Figure 11.11: Activating Auto Assign Dealer Flow

11.2 Test Drive Reminder Flow & Scheduled Paths

Creating a new Record-Triggered Flow for Test Drive Reminders on Vehicle_Test_Drive__c:

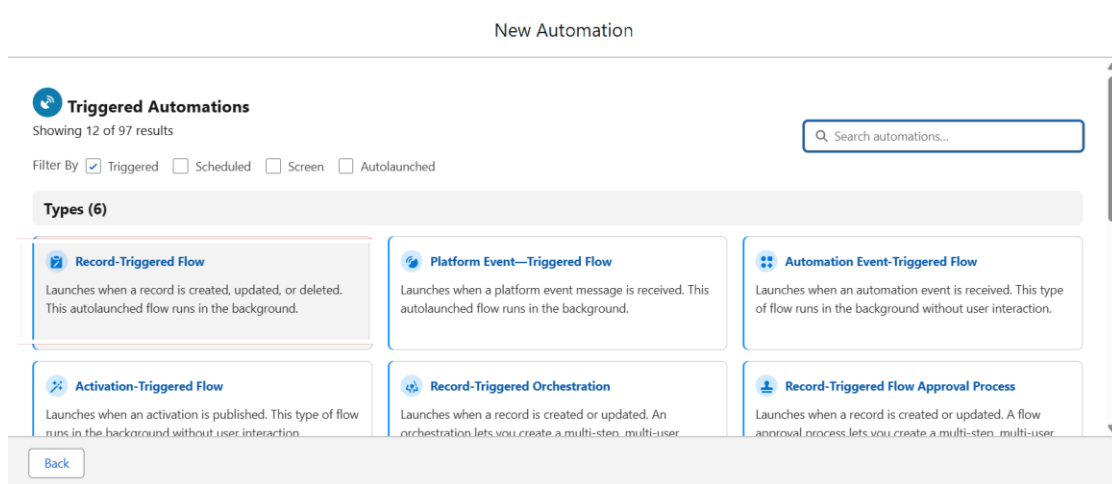


Figure 11.12: Record-Triggered Flow for Test Drive Reminder

Adding Scheduled Path below trigger:

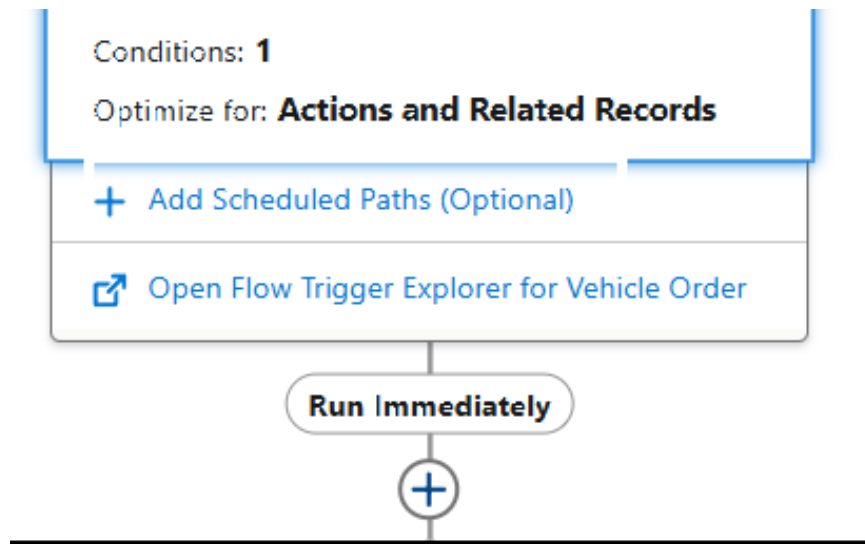


Figure 11.13: Adding Scheduled Path Option

Configuring Scheduled Path: Reminder Before Test Drive (1 Day Before Test_Drive_Date__c):

Configure Scheduled Paths

SCHEDULED PATHS +

Run Immediately

Reminder Before Test Drive.

SCHEDULED PATH DETAILS [Delete Path](#)

* Path Label:

* API Name:

* Time Source:

* Offset Number: * Offset Options:

> Show advanced options

Figure 11.14: Configuring Scheduled Path Details (1 Day Before)

Configuring Get Customer Information element in Test Drive Flow:

Get Records

* Label: * API Name:

Description:

Get Records of This Object

Object:

Filter Vehicle Customer Records

Condition Requirements:

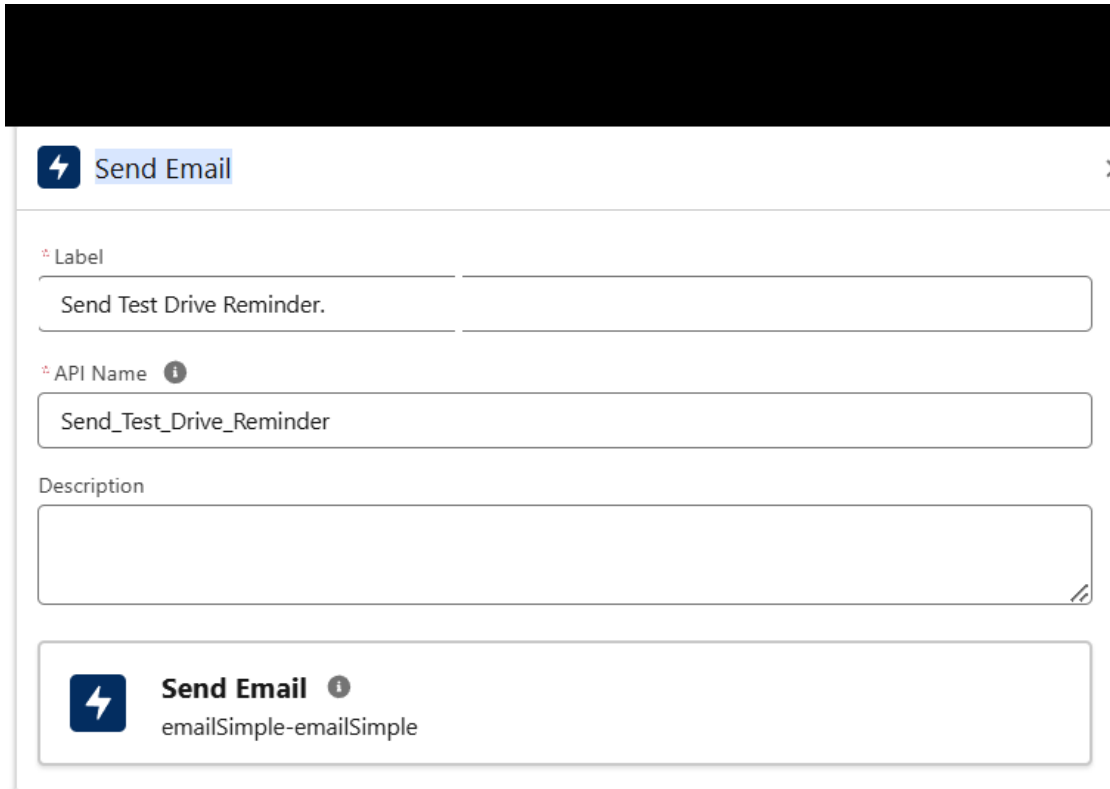
Field: Operator: Value:

+ Add Condition

Sort Vehicle Customer Records

Figure 11.15: Get Customer Information for Test Drive Reminder

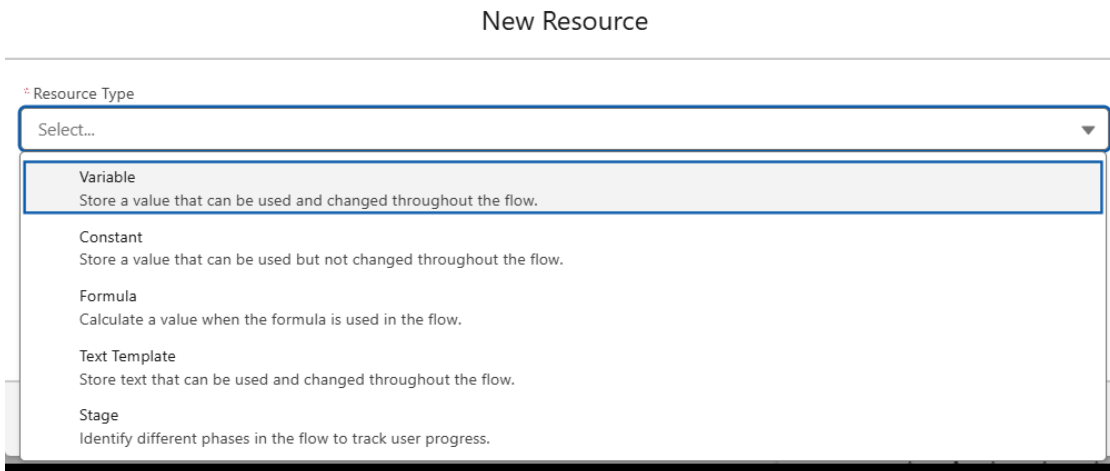
Adding Send Email Action (emailSimple) to Flow canvas:



The screenshot shows a configuration panel for a 'Send Email' action. At the top, there is a lightning bolt icon and the text 'Send Email'. Below this, there are three main sections: 'Label', 'API Name', and 'Description'. The 'Label' section has a text input field containing 'Send Test Drive Reminder.'. The 'API Name' section has a text input field containing 'Send_Test_Drive_Reminder'. The 'Description' section has a large text area. At the bottom, there is a summary box with a lightning bolt icon, the text 'Send Email', and the identifier 'emailSimple-emailSimple'.

Figure 11.16: Adding Send Email Action Element

Creating Text Template Resource for Email Body:



The screenshot shows a 'New Resource' dialog. At the top, it says 'New Resource'. Below this, there is a section titled 'Resource Type' with a dropdown menu. The dropdown menu is open, showing a list of resource types: 'Variable', 'Constant', 'Formula', 'Text Template', and 'Stage'. The 'Text Template' option is highlighted. Each option has a brief description: 'Variable' (Store a value that can be used and changed throughout the flow.), 'Constant' (Store a value that can be used but not changed throughout the flow.), 'Formula' (Calculate a value when the formula is used in the flow.), 'Text Template' (Store text that can be used and changed throughout the flow.), and 'Stage' (Identify different phases in the flow to track user progress.).

Figure 11.17: Selecting Text Template Resource Type

Editing Text Template Body (EmailSent):

Edit Text Template

* API Name ⓘ

EmailSent

Description

* Body ⓘ

Insert a resource... 🔍View as Rich Text ▼

Dear {!Get_Customer_Information.Name},

This is a reminder that your test drive for {!\$Record.Vehicle__c} is scheduled for tomorrow. If you need to reschedule, contact us at [Support Email]. Thank you, WhatNext Vision Motors Team"

Salesforce Sans ▼12 ▼■▼B I U⋮☰+≡+≡☰☷☷☷🔗🖼️I_x

CancelDone

Figure 11.18: Configuring EmailSent Rich Text Template Content

Setting Send Email action inputs (Recipient Address, Rich Text Body, Subject):

 Send Email

Body 

EmailSent

X

Included

CC Recipient Address List

Not Included

Email Template ID

Not Included

Log Email on Send

Not Included

Recipient Address Collection

Not Included

Figure 11.19: Setting Send Email Parameter Values

Saving the Flow as 'Test Drive Reminder':

Save as

Save As

A New Version

Flow Label

Test Drive Reminder

Flow API Name ⓘ

Test_Drive_Reminder

Description

[Show Advanced](#)

Cancel

Save

Figure 11.20: Saving Test Drive Reminder Flow

Activating the Test Drive Reminder Flow:

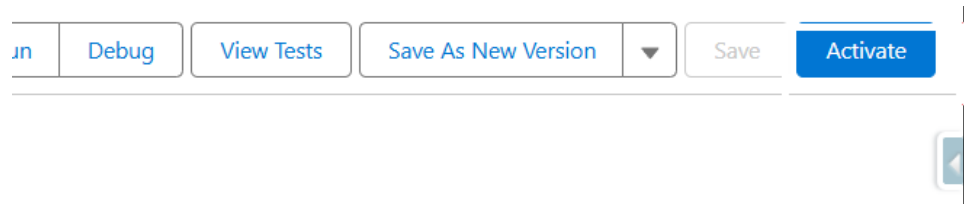


Figure 11.21: Activating Test Drive Reminder Flow

Full active Test Drive Reminder Flow canvas displaying scheduled path branch:

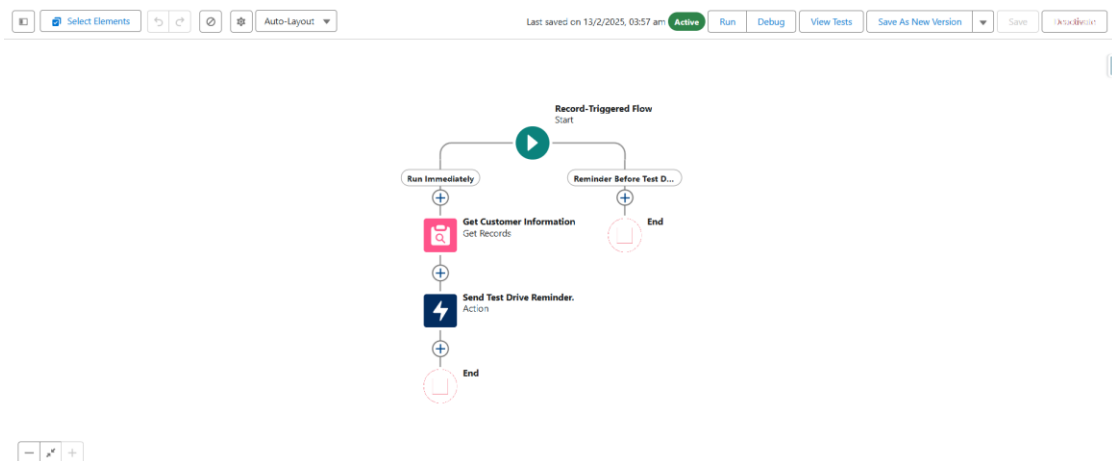


Figure 11.22: Active Test Drive Reminder Flow Canvas

Automated test drive reminder email received in Gmail inbox:

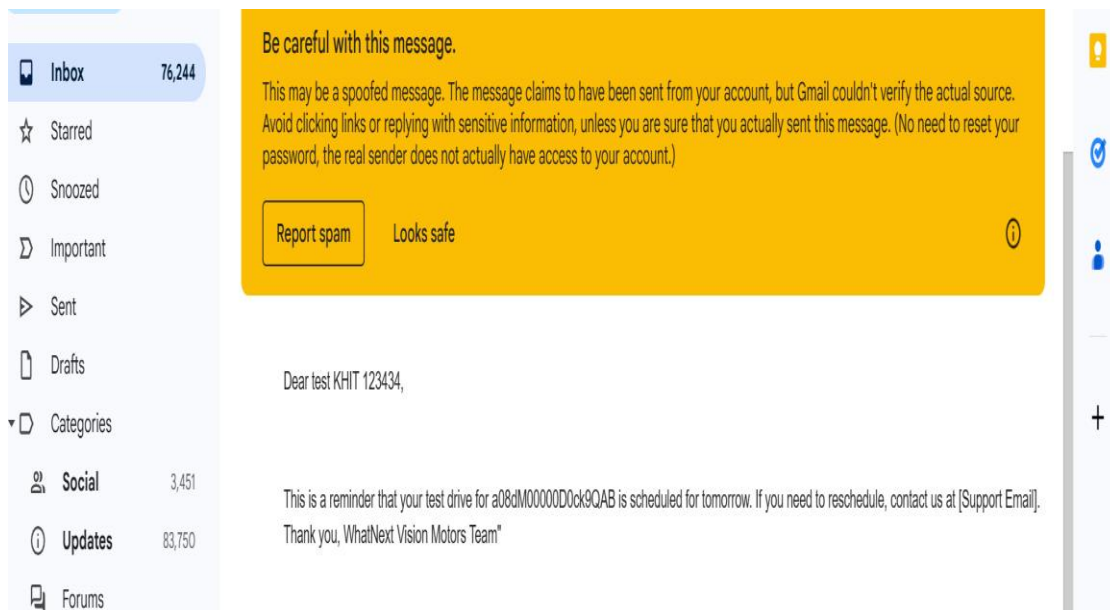


Figure 11.23: Automated Test Drive Reminder Email in Gmail Inbox

12. APEX PROGRAMMING & TRIGGER HANDLER

12.1 VehicleOrderTriggerHandler Class Implementation

Opening Developer Console from the Setup Menu:

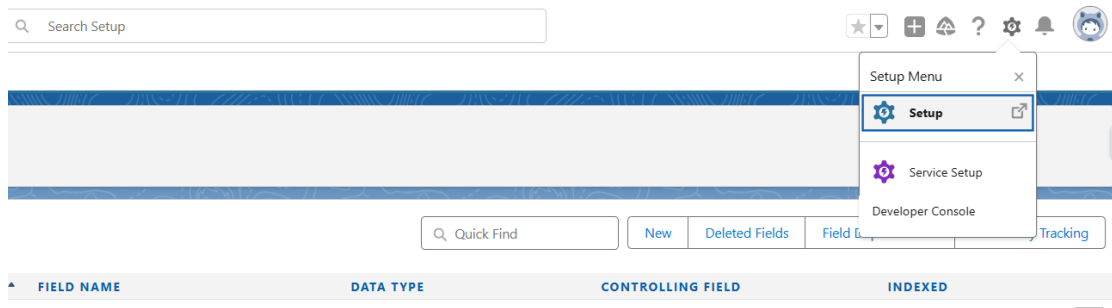


Figure 12.1: Opening Developer Console from Setup Menu

Creating a new Apex Class via File -> New -> Apex Class:

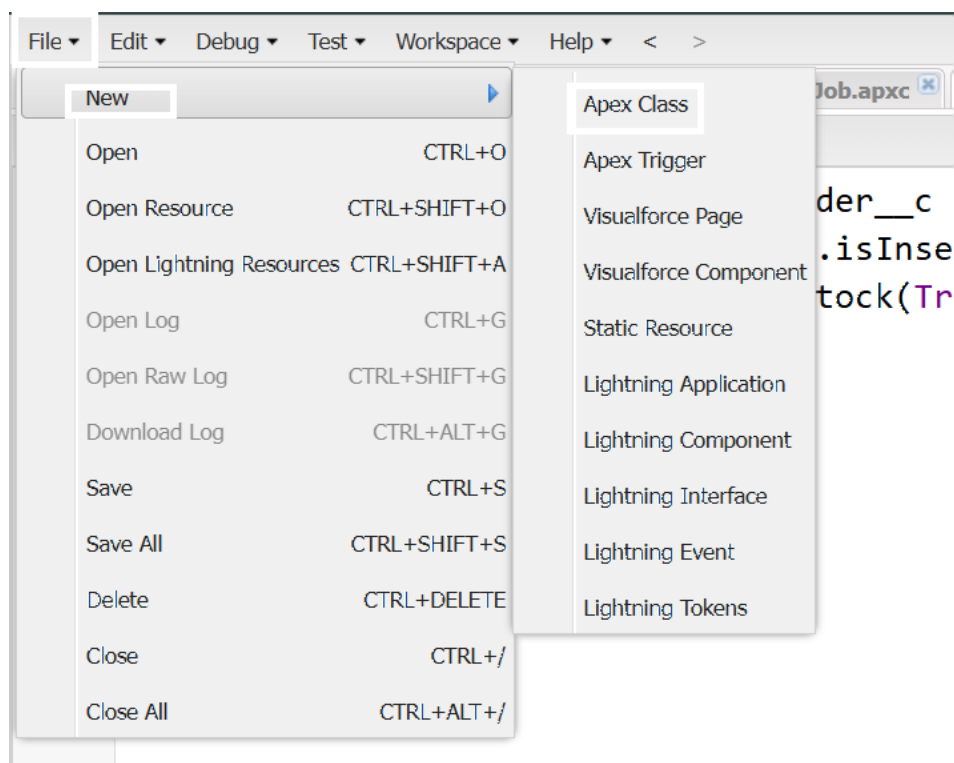


Figure 12.2: Creating New Apex Class in Developer Console

Naming the class 'VehicleOrderTriggerHandler':

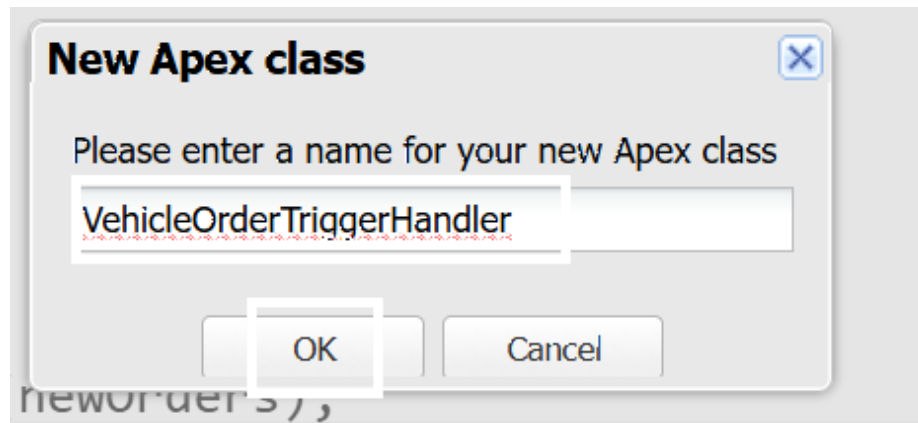


Figure 12.3: New Apex Class Dialog: VehicleOrderTriggerHandler

Writing VehicleOrderTriggerHandler code (handleTrigger and preventOrderIfOutOfStock methods):

```
1 public class VehicleOrderTriggerHandler {
2
3     public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id, Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isInsert, Boolean isUpdate) {
4         if (isBefore) {
5             if (isInsert || isUpdate) {
6                 preventOrderIfOutOfStock(newOrders);
7             }
8         }
9
10        if (isAfter) {
11            if (isInsert || isUpdate) {
12                updateStockOnOrderPlacement(newOrders);
13            }
14        }
15    }
16
17    // Method to prevent orders when the vehicle is out of stock
18    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
19        Set<Id> vehicleIds = new Set<Id>();
20        for (Vehicle_Order__c order : orders) {
21            if (order.Vehicle__c != null) {
22                vehicleIds.add(order.Vehicle__c);
23            }
24        }
25
26        if (!vehicleIds.isEmpty()) {
27            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
28            for (Vehicle__c vehicle : [SELECT Id, Stock quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
29                vehicleStockMap.put(vehicle.Id, vehicle);
30            }
31        }
32    }
33 }
```

Figure 12.4: VehicleOrderTriggerHandler Source Code (Lines 1-30)

Writing updateStockOnOrderPlacement method to decrement inventory upon confirmed orders:

```

30     }
31
32     for (Vehicle_Order__c order : orders) {
33         if (vehicleStockMap.containsKey(order.Vehicle__c)) {
34             Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
35             if (vehicle.Stock_Quantity__c <= 0) {
36                 order.addError('this vehicle is out of stock. order cannot be placed.');

```

Figure 12.5: VehicleOrderTriggerHandler Source Code (Lines 31-57)

Verifying compilation in Developer Console with zero errors:

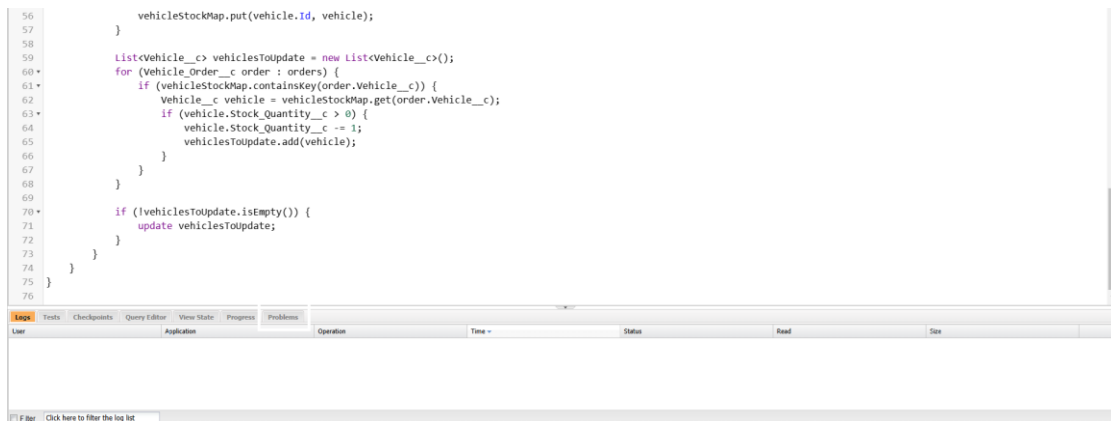


Figure 12.6: VehicleOrderTriggerHandler Compilation Verification

12.2 VehicleOrderTrigger Implementation

Creating a new Apex Trigger in Developer Console:

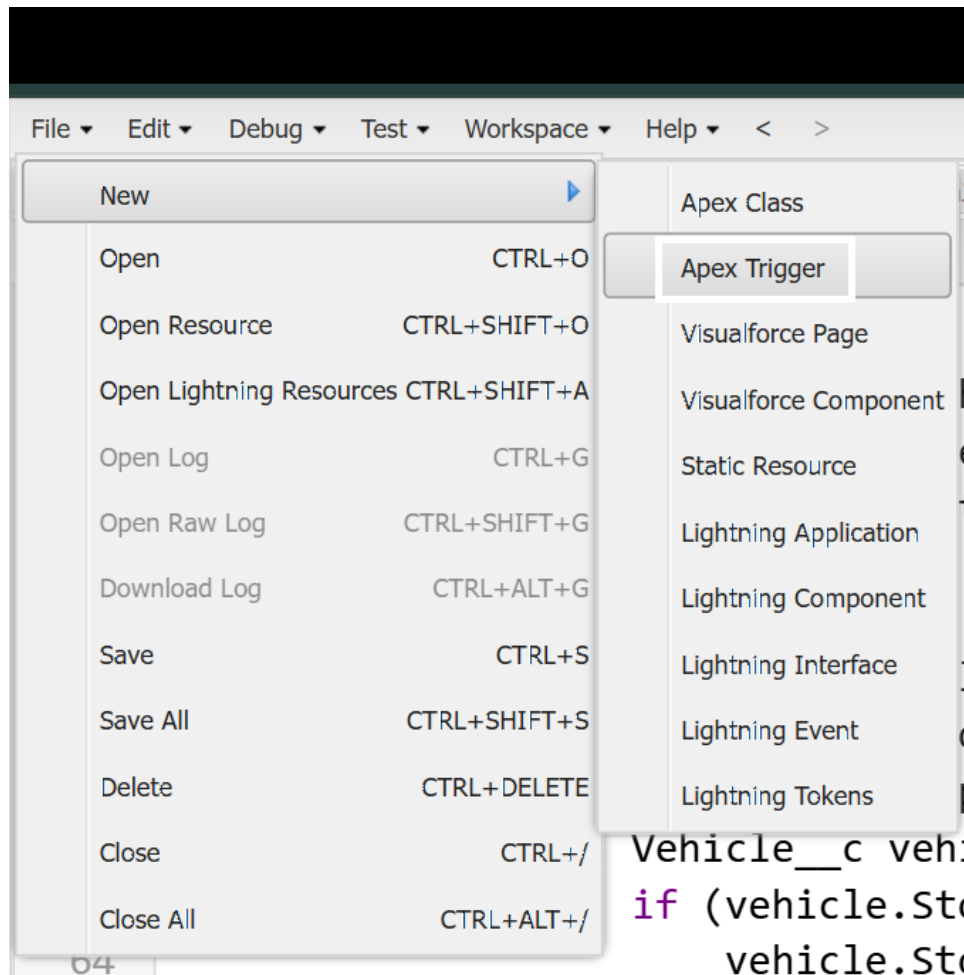
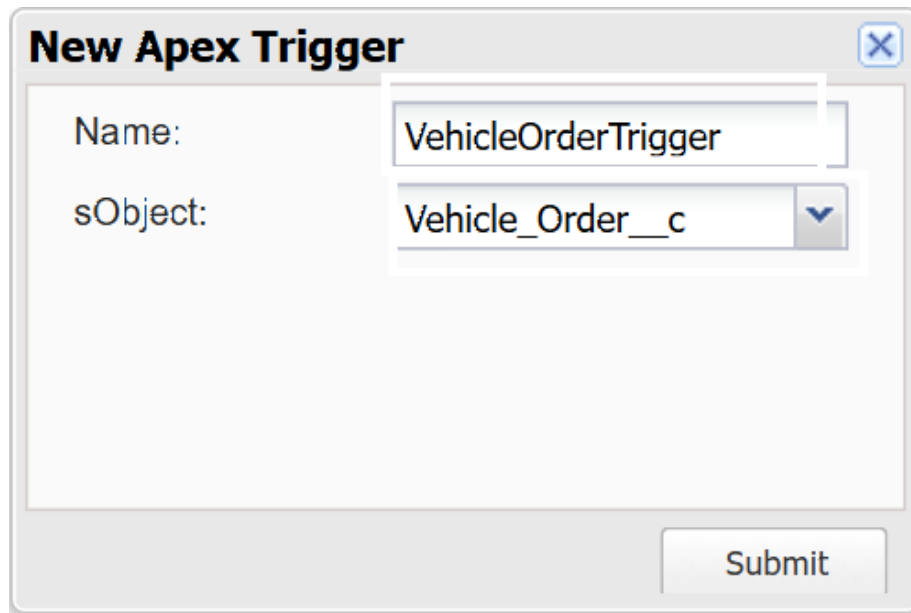


Figure 12.7: Creating New Apex Trigger in Developer Console

Configuring Trigger Name (VehicleOrderTrigger) on sObject Vehicle_Order__c:



The image shows a 'New Apex Trigger' dialog box. It has a title bar with a close button. Inside, there are two labels: 'Name:' and 'sObject:'. The 'Name:' field contains the text 'VehicleOrderTrigger'. The 'sObject:' field is a dropdown menu showing 'Vehicle_Order__c'. At the bottom right, there is a 'Submit' button.

New Apex Trigger

Name: VehicleOrderTrigger

sObject: Vehicle_Order__c

Submit

Figure 12.8: New Apex Trigger Dialog on Vehicle_Order__c

Delegating trigger events to VehicleOrderTriggerHandler:

```
Code Coverage: None | API Version: 62
1 trigger VehicleOrderTrigger on Vehicle_Order__c (before insert, before update, after insert, after update) {
2     VehicleOrderTriggerHandler.handleTrigger(trigger.new, trigger.oldMap, trigger.isBefore, trigger.isAfter, trigger.isInsert, trigger.isUpdate);
3 }
4
```

Figure 12.9: VehicleOrderTrigger Delegating Execution to Handler

12.3 Out-of-Stock Validation UI Error Trap

Attempting to order an out-of-stock vehicle. The Apex trigger catches the condition and throws the UI validation error:

The screenshot shows a web form titled "Vehicle Order VC-0003". The form contains the following fields and values:

- Vehicle Order Name: VC-0003
- Owner: Abdul Trainer
- Vehicle Customer: test KHIT 123434
- Vehicle: 1erwe
- Order Date: 21/02/2025
- Status: Pending
- Created By: Abdul Trainer, 13/02/2025, 5:34 am
- By: rainer, 13/02/2025, 5:41 am

A red error message box is displayed in the center of the form, stating: "We hit a snag. Review the errors on this page." Below this, a list of errors is shown:

- This vehicle is out of stock. Order cannot be placed.

At the bottom of the form, there are "Cancel" and "Save" buttons. A red "X" icon is visible next to the "Cancel" button.

Figure 12.10: Out-of-Stock Validation UI Error Trap in Action

13. BATCH APEX: AUTOMATED ORDER RECONCILIATION

When vehicles are restocked, pending backorders must be updated to 'Confirmed' and vehicle inventory decremented. VehicleOrderBatch implements Database.Batchable to handle high volumes without exceeding governor limits.

Creating Apex Class 'VehicleOrderBatch' in Developer Console:

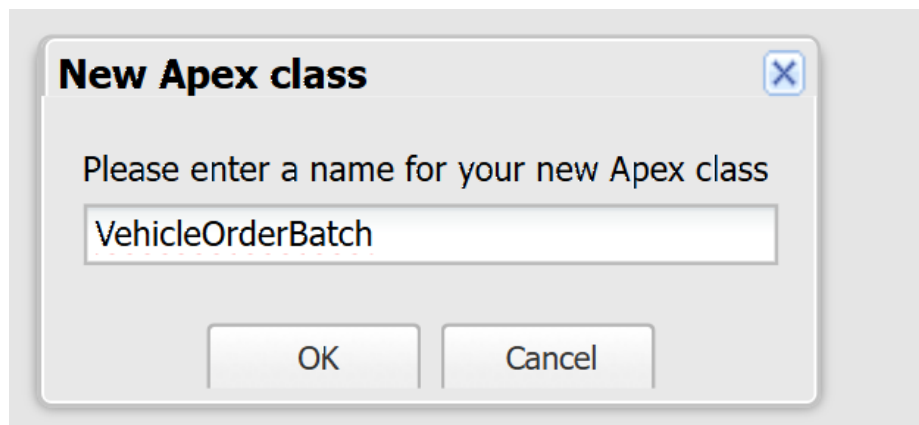


Figure 13.1: New Apex Class Dialog: VehicleOrderBatch

Implementing Database.Batchable start and execute methods:

```

Code Coverage: None | API Version: 62
1  global class VehicleOrderBatch implements Database.Batchable<sObject> {
2
3  global Database.QueryLocator start(Database.BatchableContext bc) {
4      return Database.getQueryLocator([
5          SELECT Id, Status__c, Vehicle__c
6          FROM Vehicle_Order__c
7          WHERE Status__c = 'Pending'
8      ]);
9  }
10
11 global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
12     Set<Id> vehicleIds = new Set<Id>();
13
14     for (Vehicle_Order__c order : orderList) {
15         if (order.Vehicle__c != null) {
16             vehicleIds.add(order.Vehicle__c);
17         }
18     }
19
20     if (!vehicleIds.isEmpty()) {
21         Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
22         for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
23             vehicleStockMap.put(vehicle.Id, vehicle);
24         }
25
26         List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
27         List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
28
29         for (Vehicle_Order__c order : orderList) {
30             if (vehicleStockMap.containsKey(order.Vehicle__c)) {

```

Figure 13.2: VehicleOrderBatch Source Code (start & execute methods)

Implementing execute logic and finish method in VehicleOrderBatch:

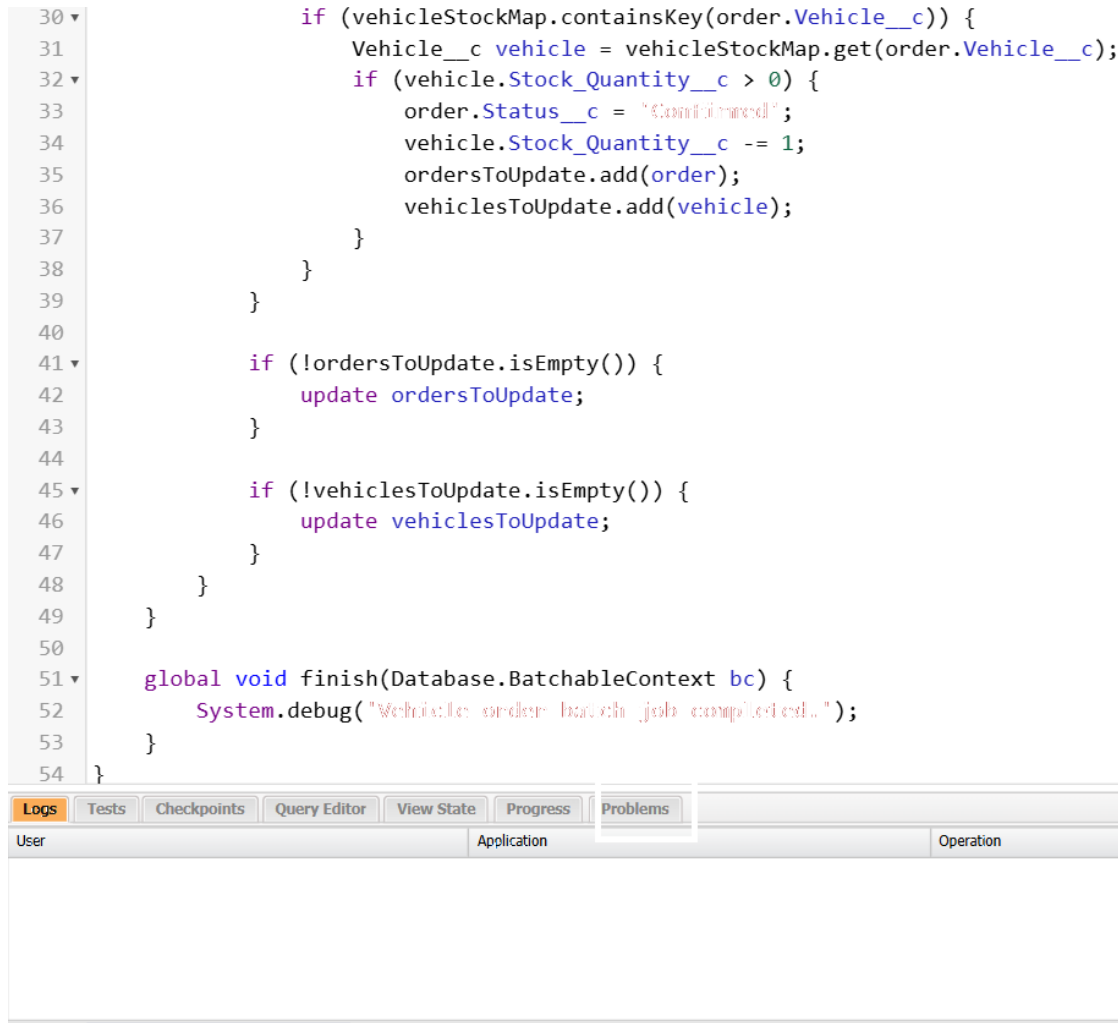


Figure 13.3: VehicleOrderBatch Finish Method and Compilation

14. SCHEDULABLE APEX & CRON AUTOMATION

To run the bulk order batch job automatically every day,
VehicleOrderBatchScheduler implements the Schedulable interface.

Creating Apex Class 'VehicleOrderBatchScheduler':

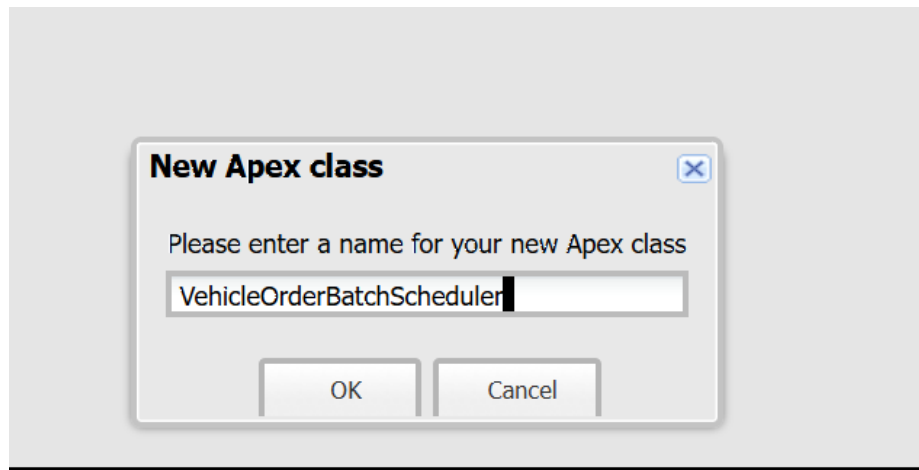


Figure 14.1: New Apex Class Dialog: VehicleOrderBatchScheduler

Implementing Schedulable execute method invoking
Database.executeBatch(batchJob, 50):

```
OrderTriggerHandler.apxc | OrderTrigger.apxt | InventoryBatchJob.apxc | VehicleOrderTriggerHandler.apxc | VehicleOrderTrigger.apxt | VehicleOrderBatchScheduler.apxc
Code Coverage: None | API Version: 62
1 global class VehicleOrderBatchScheduler implements Schedulable {
2     global void execute(SchedulableContext sc) {
3         VehicleOrderBatch batchJob = new VehicleOrderBatch();
4         Database.executeBatch(batchJob, 50);
5     }
6 }
7
```

Figure 14.2: VehicleOrderBatchScheduler Class Implementation

14.2 CRON Scheduling & Background Jobs Monitoring

Opening Execute Anonymous Window in Developer Console:

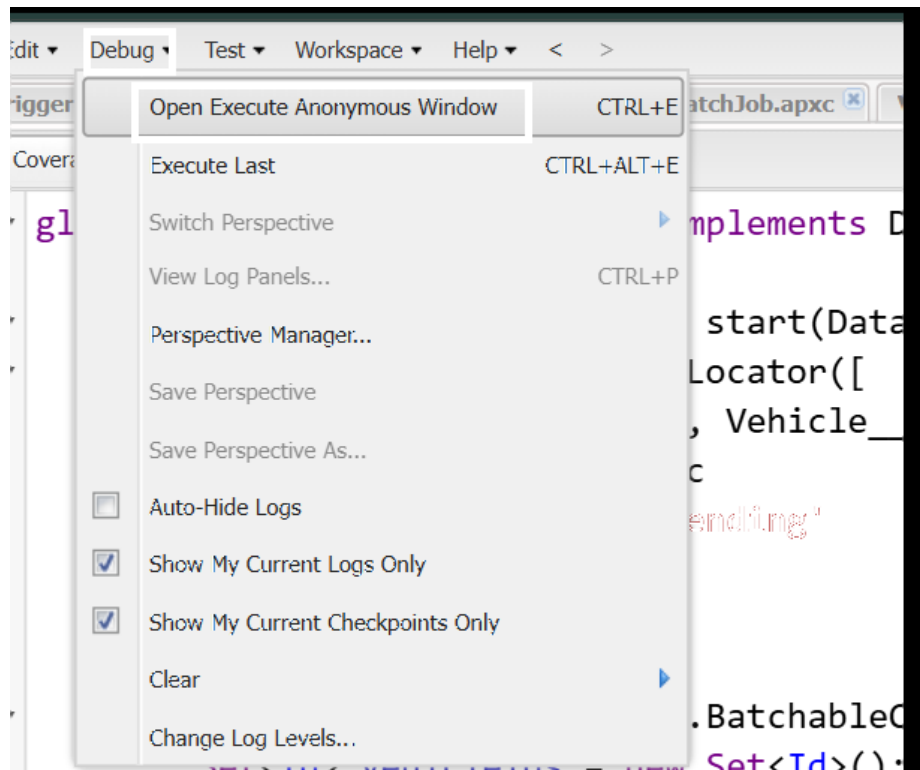
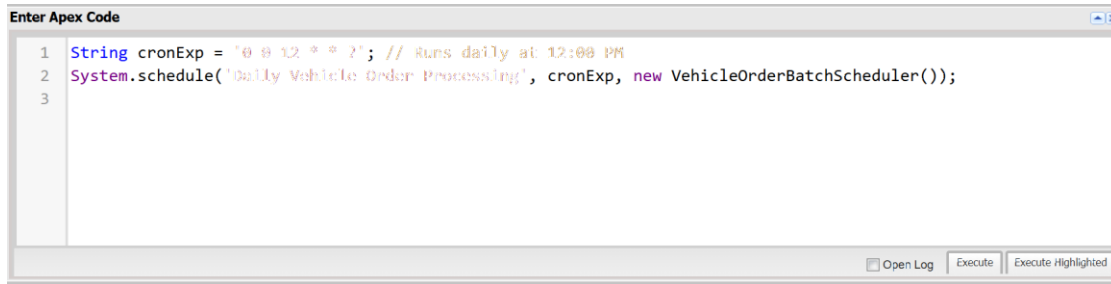


Figure 14.3: Opening Execute Anonymous Window in Developer Console

Executing CRON expression '0 0 12 * * ?' to schedule daily processing at 12:00 PM:



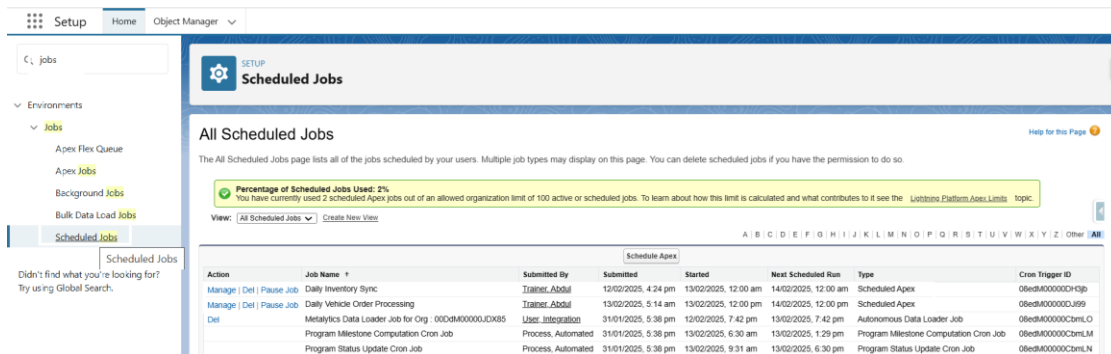
```

1 String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM
2 System.schedule('Daily Vehicle Order Processing', cronExp, new VehicleOrderBatchScheduler());
3

```

Figure 14.4: CRON Scheduling Script in Execute Anonymous

Verifying scheduled job in Setup -> Scheduled Jobs showing 'Daily Vehicle Order Processing':



Action	Job Name	Submitted By	Submitted	Started	Next Scheduled Run	Type	Cron Trigger ID
Manage Del Pause Job	Daily Inventory Sync	Trainer, Abdul	12/02/2025, 4:24 pm	13/02/2025, 12:00 am	14/02/2025, 12:00 am	Scheduled Apex	0bed4000000CH0j0
Manage Del Pause Job	Daily Vehicle Order Processing	Trainer, Abdul	13/02/2025, 5:14 am	13/02/2025, 12:00 pm	14/02/2025, 12:00 pm	Scheduled Apex	0bed4000000Cj99
Del	Metalytics Data Loader Job for Org - 000d4000000JCX85	User, Integration	31/01/2025, 5:38 pm	12/02/2025, 7:42 pm	13/02/2025, 7:42 pm	Autonomous Data Loader Job	0bed4000000CbmLO
	Program Milestone Computation Cron Job	Process, Automated	31/01/2025, 5:38 pm	13/02/2025, 6:30 am	13/02/2025, 1:29 pm	Program Milestone Computation Cron Job	0bed4000000CbmLM
	Program Status Update Cron Job	Process, Automated	31/01/2025, 5:38 pm	13/02/2025, 9:31 am	13/02/2025, 6:30 pm	Program Status Update Cron Job	0bed4000000CbmLN

Figure 14.5: Salesforce Scheduled Jobs Queue Monitor

15. TESTING STRATEGY

Test ID	Test Scenario	Expected Result	Status
TC-01	Create Vehicle record	Record created with unique Id	Passed
TC-02	Create Dealer record	Dealer record created with auto number	Passed
TC-03	Auto Assign Dealer Flow	Dealer automatically assigned from address	Passed
TC-04	Test Drive Reminder Flow	Email reminder received in Gmail	Passed
TC-05	Apex Stock Validation	Error: 'This vehicle is out of stock'	Passed
TC-06	Stock Decrement Trigger	Stock_Quantity__c decreased by 1	Passed
TC-07	VehicleOrderBatch Job	Pending orders converted to Confirmed	Passed
TC-08	VehicleOrderBatchScheduler	Job active in Scheduled Jobs queue	Passed
TC-09	CLI Deployment Dry Run	Status: Succeeded (100% components)	Passed
TC-10	GitHub Repository Sync	Working tree clean; synchronized	Passed

16. SALESFORCE CLI VALIDATION

Command: `sf project deploy start --source-dir force-app/main/default --target-org WhatNextMotorsOrg --dry-run --verbose`. Result: Status Succeeded (100% components validated).

17. GITHUB INTEGRATION

The project is hosted at: <https://github.com/Maha-2327/Salesforce.git>

Git workflow: Modify metadata -> review git status -> commit -> push to origin/main.

18. PROJECT LIMITATIONS, ADVANTAGES & FUTURE SCOPE

Advantages:

- Centralized vehicle, customer, order, and dealer records.
- Automates order routing and reduces administrative overhead.
- Prevents unfulfillable out-of-stock bookings.
- Batch Apex automatically fulfills backorders when stock is replenished.
- Traceable version history in GitHub.

Future Scope:

- Build a customer portal using Salesforce Experience Cloud.
- Incorporate Agentforce AI for conversational test drive booking and order tracking.
- Integrate Google Maps Distance Matrix API via Apex callouts.

19. CONCLUSION

WhatNext Vision Motors demonstrates a robust Salesforce automotive dealership and order management system using custom objects, Flow automation, Apex triggers, Batch Apex, and GitHub source control.

APPENDIX A: CLI COMMANDS

```
// Salesforce CLI Commands
sf --version
sf org login web --alias WhatNextMotorsOrg
sf org display --target-org WhatNextMotorsOrg
sf org list metadata --metadata-type CustomObject --target-org
WhatNextMotorsOrg
sf org list metadata --metadata-type ApexClass --target-org
WhatNextMotorsOrg
sf org list metadata --metadata-type Flow --target-org WhatNextMotorsOrg
sf project deploy start --source-dir force-app/main/default --target-org
WhatNextMotorsOrg --dry-run --verbose
```

APPENDIX B: METADATA INVENTORY

Category	Components	Count
Custom Objects	Vehicle__c, Vehicle_Dealer__c, Vehicle_Customer__c, Vehicle_Order__c, Vehicle_Test_Drive__c, Vehicle_Service_Request__c	6
Custom Fields	Stock_Quantity__c, Price__c, Vehicle_Model__c, Dealer_Location__c, Address__c, etc.	28
Custom Tabs	Vehicles, Dealers, Customers, Orders, Test Drives, Service Requests	6
Flows	Auto_Assign_Dealer, Test_Drive_Reminder	2

Apex Classes	VehicleOrderTriggerHandler, VehicleOrderBatch, VehicleOrderBatchScheduler	3
Apex Triggers	VehicleOrderTrigger	1
Applications	WhatNext_Vision_Motors	1

Respectfully submitted by:

MAHALAKSHMI U (Team Leader)

ABINAYA S (Team Member)

ELAKIYA C (Team Member)

KIRUTHIGA A (Team Member)

GOWSIK V (Team Member)

Government College of Engineering, Erode.

--- END OF PROJECT DOCUMENTATION ---